

ANA CLARA MARTINS VIEIRA (SP3120104), GUILHERME BARBOSA
CHAVES DA SILVA (SP3161706), JOÃO PAULO MOTTA SAMPAIO
(SP3128172), LARYSSA GIOVANNA MENDONÇA DE OLIVEIRA
(SP3127435), KAIQUE ZEFERINO DINIZ (SP3126811), RENAN LADISLAU
DE SOUSA (SP312651X)

CortaAi

São Paulo - SP, 2025

ANA CLARA MARTINS VIEIRA (SP3120104), GUILHERME BARBOSA
CHAVES DA SILVA (SP3161706), JOÃO PAULO MOTTA SAMPAIO
(SP3128172), LARYSSA GIOVANNA MENDONÇA DE OLIVEIRA
(SP3127435), KAIQUE ZEFERINO DINIZ (SP3126811), RENAN LADISLAU
DE SOUSA (SP312651X)

CortaAi

IFSP - Instituto Federal de Educação, Ciência e Tecnologia
Câmpus São Paulo
Tecnologia em Análise e Desenvolvimento de Sistemas

São Paulo - SP, 2025

RESUMO

Este trabalho traz a proposta do desenvolvimento de um aplicativo web de gestão de serviços e agendamentos que funcione como um *marketplace* para uma ou mais barbearias. O objetivo principal é eliminar ineficiências e desorganização, transformando a gestão do fluxo de clientes. Historicamente, muitos clientes perdem horas em longas filas para cortar o cabelo ou fazer a barba, o que demonstra que o fluxo de atendimento pode e deve ser melhorado. Essa ineficiência prejudica tanto a satisfação do cliente quanto o gerenciamento da barbearia como um negócio. O projeto surge como uma plataforma robusta para otimizar essa experiência. Além de ser um portal centralizado que facilita a procura por profissionais qualificados, horários disponíveis e serviços prestados em diferentes unidades, temos como objetivo primário automatizar o processo de agendamento. A aplicação irá gerar relatórios intuitivos e detalhados sobre desempenho, finanças e estoque, transformando a administração do negócio em algo fácil e baseado em dados. Espera-se que essa solução resulte em uma melhoria significativa na experiência do usuário (cliente e barbeiro) e na eficiência operacional das barbearias parceiras.

Palavras-chave: Gestão de Barbearia. Agendamento Online. SaaS. *Marketplace*.

SUMÁRIO

Sumário	3
1 INTRODUÇÃO	6
1.1 Objetivos	6
1.2 Problema e Solução Proposta	7
1.3 Justificativa	8
1.4 Análise da Concorrência	9
1.4.1 Salão99	9
1.4.2 AgendaServiço	10
1.4.3 eAgenda	10
1.4.4 Trinks	10
1.4.5 Comparativo	11
2 FUNDAMENTAÇÃO TEÓRICA	12
2.1 A Digitalização de serviços e experiência do cliente	12
2.2 Sistemas de gestão e agendamento para barbearias	12
3 GESTÃO DO PROJETO	14
3.1 Organização da Equipe	14
3.1.1 Responsabilidades / Papéis / Atividades	14
3.1.2 Definição de Feitos (Definition of Done - DoD)	14
3.2 Metodologias de gestão e desenvolvimento	15
3.3 Repositório da aplicação	15
4 DESENVOLVIMENTO DO PROJETO	17
4.1 Escopo do projeto	17
4.1.1 Regras de negócio	19
4.1.2 Requisitos funcionais	20
4.1.3 Requisitos Não Funcionais	23
4.2 Histórias de usuário	24
4.2.1 Criação de Barbearia (Barbeiro Dono)	24
4.2.2 Descoberta e Pesquisa (Cliente)	25
4.2.3 Gestão Avançada de Agendamentos (Cliente)	25
4.2.4 Gestão de Vínculo e Contratação (Barbeiro Dono)	25
4.2.5 Moderação de Agenda e Colaboradores (Barbeiro Dono)	26
4.2.6 Gestão de Inventário e Produtos (Barbeiro Dono)	26

4.2.7	Catálogo de Serviços e Atribuição Profissional (Barbeiro e Barbeiro Dono) .	26
4.2.8	Personalização da Jornada e Flexibilidade de Horários	27
4.3	Arquitetura da aplicação	27
4.3.1	Microserviços de Domínio	28
4.3.2	Comunicação e Persistência	29
4.3.3	Estratégia de Notificações Ativas para Redução de <i>No-Show</i>	29
4.3.4	Integração de Pagamentos Online e Fluxo Transacional	30
4.3.5	Estratégia de Engajamento e Adoção de Progressive Web Application (PWA)	31
4.4	Tecnologias	32
4.4.1	Backend e Ecosistema Spring	32
4.4.2	Mensageria e Cache	32
4.4.3	Frontend e Autenticação	33
4.4.4	Persistência e Infraestrutura	33
4.4.5	Ferramentas de Apoio	33
4.4.6	Módulo de Assistência Inteligente (Assistente Gustave)	33
4.4.7	Integração e Entrega Contínuas (CI/CD)	34
4.4.7.1	Esteira de Integração Contínua (CI)	34
4.4.7.2	Esteira de Entrega Contínua (CD)	35
4.5	Testes e manutenibilidade	36
4.5.1	Estratégia e Escopo de Teste	36
4.5.1.1	Nível 1: Testes Unitários	36
4.5.1.2	Nível 2: Testes de Integração	37
4.5.1.3	Nível 3: Testes de Componente/Integração de API	37
4.5.2	Ambientes e Ferramentas de Teste	37
4.5.3	Testes de Transição de Arquitura	38
4.5.3.1	Testes Unitários (Nível de Domínio)	38
4.5.3.2	Testes de Integração e Contratos	39
4.5.3.3	Testes de Mensageria e Eventos (Assíncronos)	39
4.5.4	Ambientes e Ferramentas de Validação	39
4.5.5	Testes funcionais	39
4.5.5.1	Testes Unitários	40
4.5.5.2	Testes de Componente	40
4.5.5.3	Testes de Integração	41
4.5.5.4	Testes end-to-end	41
4.5.5.5	Cobertura de Cenários Negativos e Casos de Borda	42
4.5.5.5.1	Cenários de Validação de Domínio (Testes Unitários)	42
4.5.5.5.2	Cenários de Restrição e Concorrência (Testes de Integração)	42
4.5.6	Testes não funcionais	43
4.5.6.1	Testes de Performance	43

4.5.6.2	Testes de Configuração	44
4.5.7	Testes automatizados	44
4.5.8	Análise estática	45
4.5.9	Code convention	45
4.5.9.1	Convenções de Backend (Java / Spring Boot)	45
4.5.9.2	Convenções de Frontend (React)	46
4.6	Segurança e Controle de Acesso	46
4.6.1	Provedor de Identidade (Firebase Authentication)	46
4.6.2	Ciclo de Vida do Token e Estratégia de Renovação	47
4.6.3	Interceção e Proteção (API Gateway)	47
4.6.4	Política de Senhas Robustas e Credenciais	48
4.6.5	Autorização Baseada em Funções (RBAC)	48
4.6.6	Observância à Legislação	49
4.7	Modelo de banco de dados	49
4.7.1	Modelo entidade relacionamento - MER	49
4.7.2	Diagrama entidade relacionamento - DER	50
4.7.2.1	Tabelas de Junção (M:N)	52
4.7.2.2	Relacionamentos Chave (Cardinalidade)	52
4.8	Cronograma do projeto	52
4.8.1	Cronograma de Sprints - Fase Inicial (2025)	53
4.8.2	Cronograma do projeto (Fase de Evolução e Estabilização)	53
4.8.3	Análise do Caminho Crítico e Riscos	54
4.9	Prototipagem e Design de Interface (UI/UX)	55
5	CONSIDERAÇÕES FINAIS	57
	REFERÊNCIAS	59
	APÊNDICE A – DICIONÁRIO DE DADOS	61

1 INTRODUÇÃO

1.1 OBJETIVOS

O setor de serviços de cuidados pessoais, embora tradicionalmente fundamentado na interação presencial, atravessa um momento de transformação digital acelerada. Os consumidores modernos exigem conveniência, personalização e, acima de tudo, previsibilidade. Modelos de negócio que dependem exclusivamente de atendimento por ordem de chegada (*walk-in*) e gestão manual estão se tornando operacionais e financeiramente insustentáveis.

O ambiente clássico das barbearias, apesar de seu valor cultural, exemplifica essa ineficiência crônica: é uma experiência comum clientes desperdiçarem horas em filas de espera, sem garantia de atendimento ou visibilidade sobre a disponibilidade de seus profissionais preferidos. Este cenário de espera e imprevisibilidade gera atrito significativo na jornada do cliente, resultando em insatisfação e perda de fidelidade. Para o gestor da barbearia, essa abordagem manual se traduz em um fluxo de trabalho caótico, dificuldade no gerenciamento das agendas dos profissionais, perda de receita de clientes que desistem da espera e uma incapacidade crônica de tomar decisões estratégicas por falta de dados consolidados. A administração do negócio torna-se reativa, focada em "apagar incêndios" operacionais, em vez de focar no crescimento.

Diante desse desafio, este trabalho propõe o desenvolvimento do CortaAi: uma aplicação web integrada, projetada para funcionar como um *marketplace* robusto para a indústria de barbearia. A solução transcende a funcionalidade de um simples aplicativo de agendamento, criando um ecossistema digital que conecta múltiplas unidades de barbearias, profissionais qualificados e clientes em uma plataforma centralizada e eficiente.

O objetivo primário do projeto é reestruturar o fluxo de atendimento, substituindo a fila física pela previsibilidade do agendamento digital. Através da plataforma, o cliente poderá pesquisar barbearias por localização, consultar o portfólio de serviços oferecidos, verificar as especialidades e a agenda de cada barbeiro, e reservar um horário (o agendamento) de forma instantânea. Para os barbeiros, o sistema oferece uma agenda digital clara, otimizando seu tempo e permitindo que se concentrem na qualidade do serviço.

Contudo, a principal inovação do CortaAi reside em sua capacidade de gestão de negócios (*Business Intelligence*). Além de organizar os agendamentos, o sistema foi modelado para integrar o controle de vendas de produtos e a gestão de estoque por unidade. Ao centralizar todas as transações, sejam elas de serviços prestados ou de produtos vendidos, a plataforma gerará relatórios intuitivos sobre o desempenho financeiro, a produtividade dos profissionais, os horários de pico e a receita por serviço. Isso transforma a administração do negócio de algo baseado na intuição para uma gestão baseada em

dados, permitindo que os proprietários otimizem seus recursos, melhorem a lucratividade e profissionalizem suas operações.

1.2 PROBLEMA E SOLUÇÃO PROPOSTA

A gestão operacional de barbearias, mesmo as que aparentam simplicidade, envolve um ecossistema complexo de processos que vão além do serviço de corte ou barba. O obstáculo central para a manutenção do negócio é a fragmentação das ferramentas de gestão. Atualmente, o processo administrativo demanda um esforço manual elevado e recorre a múltiplas ferramentas desconectadas:

- Agendas Físicas (Cadernos): Para o controle de horários, suscetíveis a rasuras, agendamentos duplicados e dificuldade de consulta remota pelos barbeiros.
- Planilhas (Excel/Google Sheets): Para o controle financeiro (fluxo de caixa) e, por vezes, para o controle de insumos, sem integração automática com as vendas.
- Notas Fiscais (Blocos de Papel ou Emissores Avulsos): Sem ligação direta com os serviços prestados ou produtos vendidos.
- Aplicativos de Mensagens (WhatsApp): Usados de forma ineficiente para o agendamento, gerando interrupções constantes e perda de informações.

Esse sistema fragmentado gera desafios operacionais críticos, como perda de informações financeiras, dificuldade na gestão de insumos (estoque), agendamentos conflitantes e ausência de métricas de desempenho.

A solução proposta é o desenvolvimento do CortaAi, uma plataforma tecnológica centralizada que visa substituir e unificar todas as tarefas de gestão fragmentadas. O núcleo do projeto será uma plataforma SaaS (*Software as a Service*) de alta disponibilidade e usabilidade, operando em um ambiente único e integrado. O sistema funcionará como o cérebro operacional da barbearia, fazendo a interface direta com:

- Módulo de Agendamentos: Permitindo a reserva online pelo cliente e a visualização da agenda em tempo real pelo barbeiro.
- Módulo Financeiro (PDV): Registrando cada serviço prestado e cada produto vendido.
- Módulo de Compras (Estoque): Controlando os insumos utilizados nos serviços e o inventário de produtos para revenda.
- Módulo de Pagamento: Integrando-se a *gateways* de pagamento para automatizar o recebimento e a conciliação financeira.

1.3 JUSTIFICATIVA

Apresentar a relevância da solução proposta, incluindo dados numéricos e gráficos que demonstrem a extensão e a importância do problema e do impacto da solução.

A importância da solução CortaAi torna-se evidente ao quantificar o custo operacional real da gestão manual para um profissional do setor. A rotina de um gestor ou "Barbeiro Dono" é intensa, totalizando aproximadamente 232 horas de trabalho mensais (considerando uma jornada de 9 horas por dia, 6 dias por semana). O problema central é que uma parcela desproporcional desse tempo é consumida por tarefas administrativas de baixo valor, que poderiam ser facilmente automatizadas.

Conforme ilustrado no Gráfico 1 (Redução do Tempo Administrativo com o Sistema), a gestão de agendamentos via métodos desorganizados—como aplicativos de mensagens (WhatsApp) e planilhas manuais—consome, em média, 40 horas por mês do profissional. Este esforço, puramente administrativo, representa 17% do tempo total de trabalho do barbeiro. Este tempo perdido equivale a quase uma semana inteira de trabalho (4.4 dias) que é desviada da atividade principal e geradora de receita: o atendimento ao cliente. Com a implementação do CortaAi, a plataforma automatiza a maior parte desse esforço, gerando um "Tempo Liberado" de aproximadamente 35 horas mensais (conforme o gráfico). A relevância do sistema reside na capacidade de transformar este tempo improdutivo em oportunidade de novos atendimentos ou gestão estratégica, liberando o profissional para focar no crescimento do negócio.

Redução do Tempo Administrativo com o Sistema

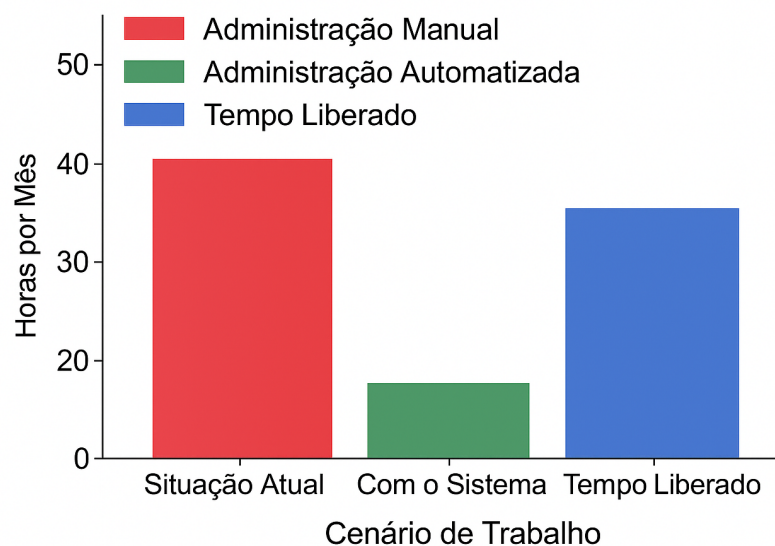


Figura 1 – Redução do Tempo Administrativo com o Sistema

Extensão e Importância da Solução: A Recuperação de Clientes A implementação do sistema de *Marketplace* para Barbearias se justifica, principalmente, pela sua capacidade

de eliminar perdas financeiras e operacionais causadas pelo fenômeno do "no-show" (não comparecimento sem aviso). A taxa de não comparecimento em agendamentos, uma dor generalizada no setor de serviços no Brasil, representa um custo de oportunidade significativo para o Barbeiro. Com uma rotina de aproximadamente 260 agendamentos mensais, o modelo de gestão manual está exposto a uma alta taxa de falha. Conforme ilustrado no Gráfico 2: Redução de *No-Shows* após Implantação do Sistema, o impacto é o seguinte:

Redução de No-Shows após Implantação do Sistema

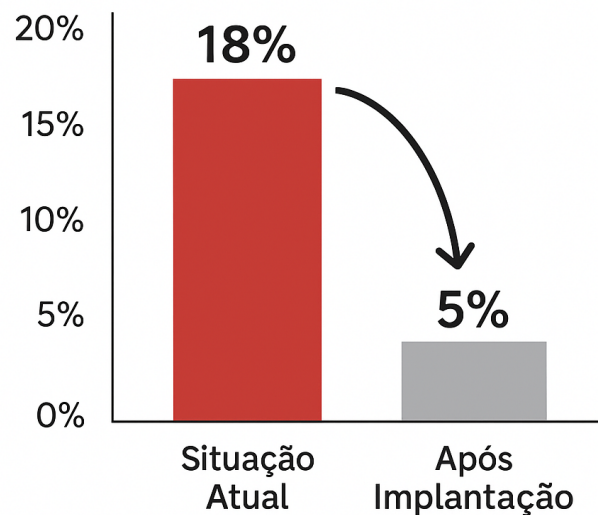


Figura 2 – Redução de *No-Shows* após Implantação do Sistema

1.4 ANÁLISE DA CONCORRÊNCIA

Avaliar os concorrentes existentes no mercado, identificando suas forças e fraquezas.

1.4.1 Salão99

O Salão99 se posiciona como uma suíte de Gestão Empresarial (ERP) para o setor de beleza, oferecendo controle financeiro avançado, gestão de comissões e fluxo de caixa. Sua Proposta de Valor (PV) reside na centralização e na profundidade dos dados gerenciais. No entanto, este escopo amplo implica, muitas vezes, em interfaces mais complexas e recursos que podem ser excessivos ou irrelevantes para as necessidades específicas e simplificadas de uma barbearia de médio ou pequeno porte. O foco em ser "completo" pode comprometer a experiência do usuário (UX) no nicho específico de barbearias.

1.4.2 AgendaServiço

O AgendaServiço adota uma Proposta de Valor de simplicidade e comunicação direta. Seu principal diferencial competitivo é a conexão otimizada via WhatsApp, que reduz a taxa de *no-show* através de lembretes automáticos e comunicações ágeis. O sistema prioriza a função de *link* personalizado de agendamento, tornando a organização de horários extremamente acessível. Contudo, essa simplificação tende a limitar a profundidade de ferramentas gerenciais, deixando de lado aspectos cruciais como o controle detalhado de custos e a emissão de relatórios financeiros customizados.

1.4.3 eAgenda

O eAgenda estrutura sua Proposta de Valor em torno da experiência multiplataforma. A existência de um aplicativo (App) dedicado para clientes e profissionais estabelece um diferencial de conveniência e engajamento. O foco está na facilidade de agendamento 24 horas e na gestão visual da agenda. De forma semelhante ao AgendaServiço, o eAgenda se destaca na dimensão Organização, mas é menos explícito ou robusto no módulo de Administração, falhando em oferecer as ferramentas necessárias para uma análise estratégica aprofundada do desempenho e rentabilidade da barbearia (ex.: margem de lucro por serviço ou taxa de reincidência por barbeiro). Estes concorrentes, embora resolvam o desafio da organização de agenda, tendem a negligenciar a Gestão Financeira e Operacional interna da barbearia. Esta insuficiência gerencial obriga os gestores a recorrerem a ferramentas externas, como planilhas, fragmentando a administração e resultando em perda de eficiência.

1.4.4 Trinks

O Trinks é uma das plataformas mais consolidadas e abrangentes no mercado brasileiro de agendamento e gestão para o setor de beleza e bem-estar. Sua Proposta de Valor baseia-se na entrega de um ecossistema completo (ERP), oferecendo desde o agendamento online multiplataforma (web e aplicativo próprio) até módulos robustos de PDV (Frente de Caixa), controle rigoroso de estoque, cálculos complexos de comissionamento e ferramentas integradas de marketing, como programas de fidelidade. Um dos seus principais diferenciais competitivos é o seu aplicativo voltado para o cliente final, que funciona como uma grande vitrine (*marketplace*). Isso facilita a descoberta de novos estabelecimentos e atrai um alto volume de usuários. Contudo, essa abrangência também é a origem de suas fraquezas em relação ao nicho específico do projeto CortaAi. Por ser uma solução genérica e desenhada para atender a toda a gama do setor de beleza (grandes salões femininos, spas, clínicas de estética e barbearias), sua interface e árvore de funcionalidades podem se tornar complexas e excessivamente poluídas para a realidade ágil e direta de uma barbearia de pequeno a médio porte. Além disso, o foco em um modelo "tudo-em-

um” para grandes estabelecimentos frequentemente se traduz em planos de assinatura com custos mais elevados, o que representa uma barreira de entrada significativa para barbeiros autônomos ou donos de barbearias em fase de crescimento que buscam uma solução focada, intuitiva e sem recursos ociosos.

1.4.5 Comparativo

Tabela 1 – Comparativo de Concorrentes

Característica	Salão99	AgendaServiço	eAgenda	Trinks	CortaAi
Foco Exclusivo (Barbearias)	Não	Não	Não	Não	Sim
Agendamento Online	Sim	Sim	Sim	Sim	Sim
Gestão Financeira Completa	Sim	Não	Não	Sim	Sim
Comissionamento	Básico	Nulo	Nulo	Avançado	Específico
Simplicidade da Interface	Média	Alta	Alta	Baixa	Alta
App Móvel Dedicado	Sim	Não	Sim	Sim	Não (Web)

2 FUNDAMENTAÇÃO TEÓRICA

Explicar os tópicos principais do projeto com base em literatura pertinente. A transição de métodos analógicos para digitais revolucionou a gestão de negócios no setor de serviços, especialmente em estabelecimentos como barbearias e salões de beleza, onde a gestão do tempo e o relacionamento com o cliente são fundamentais para o sucesso.

2.1 A DIGITALIZAÇÃO DE SERVIÇOS E EXPERIÊNCIA DO CLIENTE

Um estudo da Revista Científica da UNIFENAS (2024) aborda o potencial promissor dos salões de beleza e barbearias, que historicamente enfrentam desafios relacionados à disponibilidade de profissionais, horários e, principalmente, ao tempo de espera dos clientes. Dados coletados por esta pesquisa indicam que 67% dos clientes expressam o desejo de agendar serviços online, visando otimizar suas agendas ocupadas. A migração de um sistema de agendamento manual, sujeito a erros e desorganização, para uma plataforma digital é apresentada como uma solução direta para essa demanda. A MUTUAL TECNOLOGIA (2025) corrobora essa visão, afirmando que, para a satisfação do cliente, as inovações nos sistemas de agendamento em barbearias são imprescindíveis. A implementação de um sistema eficiente não só simplifica o processo de marcação para o cliente, mas também sinaliza a preocupação do estabelecimento em oferecer um serviço de qualidade, o que resulta em uma experiência de usuário positiva e na redução de cancelamentos de última hora.

2.2 SISTEMAS DE GESTÃO E AGENDAMENTO PARA BARBEARIAS

O SEBRAE (2023) descreve como *softwares* e aplicativos especializados facilitam e melhoram a gestão de salões. Essas ferramentas vão além do simples agendamento automatizado, que por si só já permite o controle da agenda e a reserva ou reagendamento de horários. Elas evoluem para sistemas de Gestão de Relacionamento com o Cliente (CRM), permitindo armazenar informações valiosas, como os serviços que o cliente utiliza, a frequência com que frequenta o salão e seu gasto médio. Além disso, o SEBRAE (2023) destaca a integração da agenda com módulos essenciais como "Frente de Caixa", para o controle de pagamentos (cartão, PIX, dinheiro), e "Gestão Financeira", que con-

trola o fluxo de caixa, comissões de barbeiros e a emissão de relatórios estratégicos para a tomada de decisão do proprietário.

Um artigo do IJRASET (2023) define o "Barber Shop Management System" como uma plataforma abrangente que redefine como as barbearias interagem com os clientes e gerenciam seus processos internos. O sistema une as necessidades de proprietários, funcionários e clientes. A plataforma soluciona problemas convencionais como longos tempos de espera e agendamento manual "invisível". Nestes sistemas, os usuários são comumente categorizados em "Barbeiros" (que podem gerenciar seus serviços, visualizar agendamentos e rastrear ganhos) e "Clientes" (que podem se registrar, ver horários disponíveis, escolher serviços com preços e agendar remotamente).

3 GESTÃO DO PROJETO

3.1 ORGANIZAÇÃO DA EQUIPE

A organização da equipe para abranger toda a complexidade do projeto foi pensada no tema que o aluno tem maior desenvoltura, assim agilizando o processo de criação e lapidação do projeto.

3.1.1 Responsabilidades / Papéis / Atividades

Responsabilidades, papéis e atividades de cada membro da equipe.

Tabela 2 – Responsabilidades da Equipe

Membro da Equipe	Papel	Responsabilidades
Renan Ladislau	Gerente de Projeto	Planejamento, execução, monitoramento e controle do projeto.
Renan Ladislau	Desenvolvedor Front-end	Implementação da interface do usuário.
Guilherme Chaves/Ana Clara	Desenvolvedor Back-end	Desenvolvimento da lógica de negócios e API.
João Paulo/Laryssa Giovanna	Banco de Dados	Modelagem e gerenciamento do banco de dados.
Kaique Diniz	Testes e Documentação	Testes medindo a integridade do site e suas funcionalidades assim como o registro de marcos do projeto.

3.1.2 Definição de Feitos (Definition of Done - DoD)

Para assegurar a transparência e a qualidade técnica em todos os ciclos de desenvolvimento, a equipe estabeleceu uma *Definition of Done* (DoD) rigorosa. Uma História de Usuário só é considerada "Pronta" para entrega e demonstração ao final da *Sprint* se atender integralmente aos seguintes critérios técnico-funcionais:

- **Codificação e Padrões:** O código foi desenvolvido seguindo as convenções estipuladas (Google Java Style para o *backend* e Airbnb Style para o *frontend*) e não apresenta erros de compilação.

- **Revisão de Pares (*Code Review*):** O código foi submetido via *Pull Request* no GitHub e revisado/aprovado por pelo menos um outro membro da equipe, garantindo a disseminação do conhecimento e a detecção precoce de falhas.
- **Qualidade Estática (SonarQube):** A análise estática foi concluída com sucesso, respeitando o *Quality Gate*: zero vulnerabilidades críticas e cobertura de testes em código novo superior a 80%.
- **Testes Automatizados:** Todos os testes unitários e de integração associados à funcionalidade foram executados e aprovados com sucesso no ambiente de Integração Contínua (CI).
- **Documentação Técnica:** Os contratos da API foram atualizados através de anotações do Swagger/OpenAPI, e o manual do sistema ou seções pertinentes do TCC foram devidamente revisados.
- **Validação em *Staging*:** A funcionalidade foi implantada com sucesso no ambiente de *Staging* através do *pipeline* de CD e validada funcionalmente conforme os critérios de aceitação da história.

3.2 METODOLOGIAS DE GESTÃO E DESENVOLVIMENTO

Como metodologia foi adotado o SCRUM, não de forma arbitrária, mas fundamentada na natureza e contexto de um projeto integrador que por sua vez exige não apenas a entrega de um produto funcional como também a demonstração de aprendizado e adaptabilidade a requisitos emergentes.

Adicionalmente, a estrutura de papéis do Scrum (Proprietário do Produto, Mestre Scrum e Time de Desenvolvimento) e a realização de cerimônias (Planejamento, Reunião Diária, Revisão e Retrospectiva) fomentam a transparência e a colaboração dentro da equipe. Isso é crucial para um Projeto Integrador, pois assegura que todos os membros estejam alinhados com a priorização das funcionalidades e permite a rápida identificação e resolução de impedimentos, potencializando o desempenho do time na busca pela qualidade final do produto.

Portanto, a escolha do Scrum é justificada pela sua eficácia em lidar com a complexidade inerente ao desenvolvimento de *software*, assegurando um produto que é funcional, adaptável e de valor para o nicho de barbearias.

3.3 REPOSITÓRIO DA APLICAÇÃO

O repositório da aplicação é um ponto-chave para o desenvolvimento e manutenção e para atender as demandas do mercado, juntamente com a capacitação da nossa equipe,

decidimos adotar o GitHub como forma de versionamento.

O acesso ao repositório pode ser feito via *link*, onde não há dificuldades para navegar entre as *branches*. Em prol do dinamismo no reconhecimento e facilidade em encontrar os arquivos de interesse da pessoa, nomeamos as *branches* de acordo com o desenvolvimento inerente a ela. Desta forma, ao buscar o *backend*, deve-se navegar para a branch que representa ele.

Link do repositório: https://github.com/AppCortaAi/Arquitetura_completa/tree/main

4 DESENVOLVIMENTO DO PROJETO

4.1 ESCOPO DO PROJETO

O projeto "Cortaai" consiste no desenvolvimento de um sistema de *marketplace* de barbearias, focado na automação de agendamentos e na gestão de estabelecimentos. O escopo do produto inclui as seguintes funcionalidades, suportadas por um *backend* robusto em Java/Spring Boot:

1. Módulo de Autenticação e Usuários (Cliente e Barbeiro):

- Registro de Clientes e Barbeiros com validação de campos únicos (e-mail, telefone, CPF) e formato (CPF).
- Login seguro para ambos os tipos de usuário usando e-mail e senha, com criptografia.
- Autenticação baseada em Firebase Auth, definindo papéis (`ROLE_CUSTOMER` , `ROLE_BARBER` , `ROLE_OWNER`).
- Gerenciamento de perfil (atualização de dados cadastrais) para Clientes e Barbeiros autenticados.

2. Módulo de Gerenciamento de Barbearia (Dono):

- Um Barbeiro pode registrar uma nova barbearia, tornando-se Barbeiro Owner .
- Upload de mídia (Logo, Banner, Imagens de Destaque) para o perfil da barbearia, com armazenamento no Cloudinary.
- Cadastro de serviços oferecidos pela barbearia, incluindo nome, preço, duração e foto do serviço.

3. Módulo de Gerenciamento de Equipe (Barbeiro e Dono):

- Barbeiro pode solicitar entrada em uma barbearia existente via CNPJ.
- Barbeiro_Owner pode visualizar, aprovar ou rejeitar solicitações de entrada pendentes de novos barbeiros na barbearia.
- Barbeiro pode definir seu horário de trabalho (início e fim).
- Barbeiro pode vincular a seu perfil quais serviços da barbearia ele está apto a realizar.
- Barbeiro pode consultar seu histórico de solicitações de entrada.

4. Módulo de Agendamento (Core):

- Cálculo de disponibilidade de horários para um barbeiro, com base em seu horário de trabalho e agendamentos existentes.
- Criação e atualização de agendamentos pelo Cliente, com validação de regras de negócio.
- Cancelamento de agendamentos pelo Cliente (dono do agendamento) ou pelo Dono da barbearia.
- Consulta de agendas (listagem de agendamentos) com visões segregadas por papel.

5. Descoberta

- *Endpoints* públicos para listar barbearias, seus serviços e seus barbeiros, permitindo a descoberta pelo cliente.

Premissas

O sucesso do projeto baseia-se nas seguintes premissas:

- **Infraestrutura Disponível:** Assume-se que a infraestrutura de implantação baseada em contêineres, bem como os serviços de banco de dados (MySQL e Redis) e de integrações de terceiros, estarão disponíveis e operacionais com alta disponibilidade.
- **Credenciais Válidas:** Assume-se que as credenciais de acesso aos bancos de dados, *message brokers* e serviços externos são rigorosamente válidas e seguras.
- **Consumo da API:** Assume-se que a aplicação *frontend* (desenvolvida em React e Vite, e operando como PWA) funcionará como cliente exclusivo, comunicando-se de forma assíncrona com os domínios internos através do *API Gateway*.
- **Conectividade:** Assume-se que os usuários finais terão acesso à internet de forma minimamente estável para garantir a fluidez da *Single Page Application* (SPA).

Restrições

O projeto está sujeito às seguintes restrições:

- **Tecnologia Backend:** O desenvolvimento está restrito ao ecossistema Java 17 e Spring Boot (versão 3.x), adotando obrigatoriamente o padrão arquitetural de microsserviços com o suporte do Spring Cloud (Eureka Discovery e Spring Cloud Gateway).

- **Persistência e Cache:** O sistema deve utilizar o banco de dados MySQL para a persistência relacional das entidades e transações, atuando em conjunto com o Redis para estratégias de *caching* em operações críticas (como consultas de disponibilidade de agendas).
- **Plataforma de Implantação:** A arquitetura de implantação deve ser containerizada (via Docker), substituindo em caráter definitivo o design *serverless* adotado nas prototipações iniciais do monólito.
- **Armazenamento de Mídia:** O armazenamento, a manipulação e a entrega de mídia (como logótipos e imagens do portfólio de cortes) estão estritamente vinculados ao serviço Cloudinary.
- **Segurança:** O modelo de segurança implementa autenticação *stateless* baseada em *tokens* JWT emitidos pelo provedor de identidade **Firebase Authentication**. A validação ocorre de forma coordenada na malha: o *API Gateway* atua na filtragem de borda validando a integridade inicial da requisição, enquanto os filtros de segurança individuais (**JwtAuthorizationFilter**) de cada microserviço de domínio decodificam o *token* de forma descentralizada para aplicar o controle de acesso baseado em funções.

4.1.1 Regras de negócio

Descrever as regras que governam as operações do sistema.

Tabela 3 – Regras de Negócio

Código	Descrição	Relacionados
RN1	"Usuários não autenticados podem acessar <i>endpoints</i> públicos (listagens gerais, registro, login, Swagger)."	
RN2	"Autenticação é necessária para acessar funcionalidades específicas de Cliente, Barbeiro ou Dono."	
RN3	"A autenticação é realizada via e-mail e senha, retornando um token JWT."	RN2
RN4	O token JWT deve ser enviado no cabeçalho Authorization: Bearer {token} para acessar rotas protegidas.	"RN2, RN3"
RN5	Apenas o Cliente pode gerenciar seu próprio perfil e agendamentos.	RN2
RN6	"Apenas o Barbeiro pode gerenciar seu próprio perfil, solicitar entrada ou sair de uma loja."	RN2
RN7	Apenas o Barbeiro Owner pode gerenciar sua barbearia.	"RN2, RN6"

Tabela 3: Continuação

Código	Descrição	Relacionados
RN8	Um Barbeiro só pode se tornar Barbeiro Owner ao registrar a sua primeira barbearia.	"RN6, RN7"
RN9	"E-mail, telefone e CPF devem ser únicos para Clientes e Barbeiros."	
RN10	CPF e CNPJ devem seguir o formato e algoritmo de validação brasileiros.	
RN11	"Um agendamento requer um Cliente, Barbeiro, Barbearia e pelo menos um Serviço válido."	
RN12	O Barbeiro selecionado para o agendamento deve pertencer à Barbearia selecionada.	RN11
RN13	Os Serviços selecionados para o agendamento devem pertencer à Barbearia selecionada.	RN11
RN14	O Barbeiro selecionado deve estar habilitado a realizar todos os Serviços selecionados.	"RN11, RN13"
RN15	O horário de início do agendamento deve ser no futuro.	RN11
RN16	O horário do agendamento deve estar dentro do horário de trabalho definido para o Barbeiro.	RN11
RN17	O horário do agendamento não pode conflitar com outros agendamentos existentes para o mesmo Barbeiro (exceto cancelados).	"RN11, RN16"
RN18	"Um Cliente ou o Dono da Barbearia podem cancelar um agendamento, desde que não esteja concluído."	"RN5, RN7"
RN19	Apenas o Dono pode excluir fisicamente um agendamento.	RN7
RN20	Um Barbeiro só pode solicitar entrada em uma barbearia se não estiver vinculado a nenhuma.	RN6
RN21	Um Dono só pode aprovar uma solicitação se o Barbeiro ainda estiver livre.	"RN7, RN20"
RN22	O Dono pode rejeitar uma solicitação.	RN7

4.1.2 Requisitos funcionais

Listar os requisitos que descrevem as funcionalidades que o sistema deve oferecer.

Tabela 4 – Requisitos Funcionais

Código	Descrição	Relacionados
RF1	O sistema deve permitir o registro de novos Clientes.	”RN9, RN10”
RF2	O sistema deve permitir o registro de novos Barbeiros.	”RN9, RN10”
RF3	”O sistema deve permitir o login de Clientes via e-mail e senha, retornando um token JWT.”	”RN3, RN4”
RF4	”O sistema deve permitir o login de Barbeiros via e-mail e senha, retornando um token JWT.”	”RN3, RN4”
RF5	Um Cliente autenticado deve poder visualizar e atualizar seus dados cadastrais (exceto senha).	”RN5, RN9, RN10”
RF6	Um Barbeiro autenticado deve poder visualizar e atualizar seus dados cadastrais (exceto senha).	”RN6, RN9, RN10”
RF7	Um Cliente autenticado deve poder fazer upload/atualizar sua foto de perfil.	RN5
RF8	Um Barbeiro autenticado deve poder fazer upload/atualizar sua foto de perfil.	RN6
RF9	Qualquer usuário (autenticado ou não) deve poder listar todas as Barbearias cadastradas.	RN1
RF10	Qualquer usuário deve poder listar os Serviços oferecidos por uma Barbearia específica.	RN1
RF11	Qualquer usuário deve poder listar os Barbeiros que trabalham em uma Barbearia específica.	RN1
RF12	”Um Barbeiro autenticado (sem loja) deve poder registrar sua primeira Barbearia, tornando-se Dono.”	”RN6, RN8, RN10”
RF13	”Um Dono autenticado deve poder atualizar os dados da sua Barbearia (nome, endereço).”	RN7
RF14	Um Dono autenticado deve poder fazer upload/atualizar o logo da sua Barbearia.	RN7
RF15	Um Dono autenticado deve poder fazer upload/atualizar o banner da sua Barbearia.	RN7
RF16	Um Dono autenticado deve poder adicionar imagens de destaque à sua Barbearia.	RN7

Tabela 4: Continuação

Código	Descrição	Relacionados
RF17	Um Dono autenticado deve poder remover imagens de destaque da sua Barbearia.	RN7
RF18	Um Dono autenticado deve poder cadastrar novos Serviços para sua Barbearia.	RN7
RF19	Um Dono autenticado deve poder fazer upload/atualizar a foto de um Serviço.	RN7
RF20	Um Barbeiro autenticado (sem loja) deve poder solicitar entrada em uma Barbearia existente via CNPJ.	"RN6, RN20"
RF21	Um Dono autenticado deve poder visualizar as solicitações de entrada pendentes para sua Barbearia.	RN7
RF22	Um Dono autenticado deve poder aprovar uma solicitação de entrada.	"RN7, RN21"
RF23	Um Dono autenticado deve poder rejeitar uma solicitação de entrada.	"RN7, RN22"
RF24	Um Barbeiro autenticado (não-dono) deve poder sair da Barbearia à qual está vinculado.	RN6
RF25	Um Dono autenticado deve poder remover um Barbeiro (não-dono) da sua Barbearia.	RN7
RF26	Um Barbeiro autenticado deve poder definir seu horário de trabalho.	RN6
RF27	Um Barbeiro autenticado (vinculado a uma loja) deve poder definir quais Serviços da loja ele realiza.	RN6
RF28	Qualquer usuário deve poder consultar os horários disponíveis de um Barbeiro para uma data e duração específicas.	"RN1, RN16, RN17"
RF29	Um Cliente autenticado deve poder criar um novo Agendamento.	"RN5, RN11, RN12, RN13, RN14, RN15, RN16, RN17"
RF30	Um Cliente autenticado deve poder atualizar um Agendamento existente (não concluído/cancelado).	"RN5, RN11-RN17"

Tabela 4: Continuação

Código	Descrição	Relacionados
RF31	Um Cliente autenticado deve poder cancelar um Agendamento seu (não concluído).	"RN5, RN18"
RF32	Um Dono autenticado deve poder cancelar qualquer Agendamento da sua loja (não concluído).	"RN7, RN18"
RF33	Um Dono autenticado deve poder excluir fisicamente um Agendamento da sua loja.	"RN7, RN19"
RF34	Um Cliente autenticado deve poder consultar a lista dos seus Agendamentos.	RN5
RF35	Um Barbeiro autenticado deve poder consultar a lista dos seus Agendamentos.	RN6
RF36	Um Dono autenticado deve poder consultar a lista de todos os Agendamentos da sua loja.	RN7
RF37	Um Barbeiro autenticado deve poder consultar o histórico das suas solicitações de entrada.	RN6
RF38	O sistema deve prover documentação da API via Swagger UI.	RN1

4.1.3 Requisitos Não Funcionais

Os Requisitos Não Funcionais (RNF) definem as premissas de qualidade, segurança, desempenho e infraestrutura que o ecossistema CortaAi deve obrigatoriamente respeitar. Com a evolução tática para uma arquitetura distribuída, estes requisitos foram reavaliados para garantir alta disponibilidade e tolerância a falhas num ambiente de nuvem moderno.

- **[RNF01] Segurança e Gestão de Identidade Federada:** A plataforma deve assegurar a máxima proteção dos dados dos utilizadores. Para mitigar riscos de violação de dados, o sistema não armazenará palavras-passe localmente, delegando a autenticação ao **Firestore Authentication**. Todo o tráfego de entrada será estritamente interceptado, validado criptograficamente (via JWT) e roteado de forma segura através de um *API Gateway* central.
- **[RNF02] Escalabilidade Independente e Contendorização:** O sistema deve ser capaz de suportar flutuações e picos de utilização sem degradação do serviço. A infraestrutura deve basear-se em microsserviços empacotados isoladamente em contentores **Docker**. Isto garantirá que domínios sob maior *stress* (como o módulo de agendamentos) possam ser escalados horizontalmente de forma totalmente independente dos restantes serviços.

- **[RNF03] Desempenho e Otimização de Leitura:** O tempo de resposta da interface deve ser fluído e imediato, proporcionando uma excelente experiência de utilização (UX). Para evitar sobrecargas no banco de dados relacional durante consultas frequentes (como a pesquisa de horários livres dos barbeiros), o sistema deverá implementar estratégias de *cache* distribuído em memória através do **Redis**.
- **[RNF04] Resiliência e Processamento Assíncrono:** A plataforma não deve bloquear o cliente enquanto processa tarefas pesadas. Operações secundárias, como o disparo de e-mails, alertas de *stock* baixo ou registos de pagamentos, devem ser executadas em segundo plano. O sistema adotará uma Arquitetura Orientada a Eventos, utilizando o **RabbitMQ** como *Message Broker*, para assegurar a consistência eventual dos dados e a alta tolerância a falhas de comunicação.
- **[RNF05] Isolamento de Dados (*Database-per-Service*):** Para garantir o baixo acoplamento, uma falha crítica no banco de dados de um serviço não deve comprometer a totalidade da plataforma. O sistema deve adotar o padrão de persistência isolada (bases de dados MySQL dedicadas por microserviço), comunicando-se inter-serviços unicamente através de contratos REST (OpenFeign) ou mensageria.

4.2 HISTÓRIAS DE USUÁRIO

As Histórias de Usuário foram definidas com base nos casos de uso para os três atores principais do sistema: Cliente, Barbeiro e Barbeiro Dono.

4.2.1 Criação de Barbearia (Barbeiro Dono)

Eu como um Barbeiro Dono de uma barbearia, quero registar os dados fundamentais do meu estabelecimento (nome, CNPJ, endereço, horário padrão de funcionamento e identidade visual), para que a minha barbearia passe a existir na plataforma e eu possa prosseguir com a configuração do catálogo de serviços e o vínculo da minha equipe de barbeiros.

Critérios de Aceitação:

- **Critério 1:** O proprietário deve poder preencher e submeter o formulário de registo da barbearia, incluindo o anexo do logótipo do estabelecimento.
- **Critério 2:** O sistema deve validar o CNPJ inserido, bloqueando o cadastro e exibindo uma mensagem de erro caso o documento seja inválido ou já se encontre registado por outra barbearia.
- **Critério 3:** Após o sucesso na submissão e gravação dos dados, o sistema deve redirecionar o proprietário de imediato para a etapa de gestão e criação de serviços.

4.2.2 Descoberta e Pesquisa (Cliente)

Eu como um Visitante ou Cliente não autenticado, quero pesquisar barbearias, visualizar os serviços oferecidos e os profissionais disponíveis em cada estabelecimento, para decidir onde e com quem agendar o meu atendimento.

Critérios de Aceitação:

- **Critério 1:** O cliente deve poder listar todas as barbearias registradas no sistema sem necessidade de *login* (Cobre RF9).
- **Critério 2:** Ao selecionar uma barbearia, o sistema deve exibir a lista de serviços (com preço e duração) oferecidos (Cobre RF10).
- **Critério 3:** O sistema deve permitir a visualização da equipe de barbeiros vinculada à barbearia selecionada (Cobre RF11).

4.2.3 Gestão Avançada de Agendamentos (Cliente)

Eu como Cliente autenticado, quero poder consultar, reagendar ou cancelar os meus agendamentos futuros, para ter flexibilidade caso ocorra um imprevisto na minha rotina.

Critérios de Aceitação:

- **Critério 1:** O Cliente deve poder visualizar uma lista com o histórico e os agendamentos futuros associados ao seu perfil (Cobre RF34).
- **Critério 2:** O Cliente deve poder reagendar (atualizar data/hora) um agendamento, desde que o mesmo não esteja com o estado "Concluído" ou "Cancelado" (Cobre RF30).
- **Critério 3:** O Cliente deve poder cancelar um agendamento ativo, libertando o horário na agenda do barbeiro (Cobre RF31).

4.2.4 Gestão de Vínculo e Contratação (Barbeiro Dono)

Eu como Barbeiro Dono, quero vincular um barbeiro profissional à minha equipe através do seu CPF, para que o processo de contratação seja iniciado pela empresa, prevenindo o excesso de notificações de solicitações externas na gestão administrativa da barbearia.

Critérios de Aceitação:

- **Critério 1:** O Dono da barbearia deve poder convidar um barbeiro para a sua equipe informando o CPF do profissional já devidamente registrado na plataforma (Cobre RF20)
- **Critério 2:** O Barbeiro convidado deve poder visualizar os convites pendentes e optar por aceitar ou rejeitar o vínculo com o estabelecimento (Cobre RF37)

- **Critério 3:** O sistema deve permitir que o Barbeiro se desvincule da barbearia atual de forma autónoma, garantindo a sua liberdade de atuação profissional (Cobre RF24)

4.2.5 Moderação de Agenda e Colaboradores (Barbeiro Dono)

Eu como Barbeiro Dono, quero ter o controle absoluto sobre a agenda da minha loja e a minha equipe, podendo forçar cancelamentos ou remover profissionais, para lidar com imprevistos operacionais ou encerramentos do espaço.

Critérios de Aceitação:

- **Critério 1:** O Dono deve poder cancelar administrativamente qualquer agendamento da sua loja, independentemente do cliente ou barbeiro alocado (Cobre RF32).
- **Critério 2:** Em casos extremos (ex.: fraude ou erro), o Dono deve poder excluir fisicamente um agendamento do banco de dados (Cobre RF33).
- **Critério 3:** O Dono deve poder remover da equipe um barbeiro (não-dono) que não preste mais serviços no seu estabelecimento (Cobre RF25).

4.2.6 Gestão de Inventário e Produtos (Barbeiro Dono)

Eu como Barbeiro Dono, quero cadastrar, visualizar e alterar os produtos referentes ao estoque da minha unidade, para que eu possa ter o controlo total sobre os suprimentos e vendas de itens físicos de forma centralizada.

Critérios de Aceitação:

- **Critério 1:** O acesso às funcionalidades de criação, edição e visualização de inventário deve ser restrito exclusivamente a utilizadores com a função `ROLE_OWNER`.
- **Critério 2:** O sistema deve permitir o registo de novos produtos informando nome, preço de venda, quantidade em *stock* e descrição.
- **Critério 3:** O Dono deve poder atualizar a quantidade em estoque e os valores dos produtos a qualquer momento, refletindo as alterações no inventário em tempo real.

4.2.7 Catálogo de Serviços e Atribuição Profissional (Barbeiro e Barbeiro Dono)

Eu como Barbeiro Dono, quero definir o menu de serviços da barbearia e, como Barbeiro, quero selecionar quais desses serviços estou apto a realizar, para que a agenda de agendamentos reflita com precisão as competências de cada profissional da equipa.

Critérios de Aceitação:

- **Critério 1:** O Barbeiro Dono deve poder criar e configurar serviços definindo o nome, o preço e a duração estimada (Cobre RF18).
- **Critério 2:** Um Barbeiro, após ter o seu vínculo aceito na barbearia, deve ter acesso à lista completa de serviços previamente cadastrados pelo dono.
- **Critério 3:** O Barbeiro deve poder selecionar individualmente quais os serviços do catálogo da loja ele realiza, vinculando-os ao seu perfil profissional (Cobre RF27).
- **Critério 4:** O sistema de agendamento só deve permitir que um Cliente reserve um serviço específico com um Barbeiro se este profissional tiver selecionado previamente tal atividade como parte do seu portfólio.

4.2.8 Personalização da Jornada e Flexibilidade de Horários

Eu como Barbeiro, quero ter a autonomia para selecionar quais dias da semana estarei disponível para atendimento e definir turnos específicos com horários espaçados, para que minha agenda reflita fielmente minha disponibilidade real e permita intervalos de descanso ou compromissos externos.

Critérios de Aceitação:

- **Critério 1:** O sistema deve permitir que o Barbeiro selecione individualmente os dias da semana (ex.: segunda a sábado) em que prestará serviços na unidade.
- **Critério 2:** O Barbeiro deve poder configurar múltiplos intervalos de tempo para um mesmo dia (ex.: das 09:00 às 12:00 e das 14:00 às 19:00), suportando jornadas com horários espaçados.
- **Critério 3:** O motor de busca de disponibilidade do *Schedule Service* deve considerar estes intervalos para não exibir horários vagos durante os períodos de pausa definidos pelo profissional.
- **Critério 4:** Qualquer alteração na jornada de trabalho deve ser validada contra os agendamentos já existentes, impedindo conflitos de horários.

4.3 ARQUITETURA DA APLICAÇÃO

Com a evolução do projeto e a necessidade de maior escalabilidade, resiliência e independência entre os domínios de negócio, a arquitetura do sistema transitou de um modelo monolítico para uma **Arquitetura de Microserviços Orientada a Eventos** (*Event-Driven Microservices Architecture*).

O frontend (desenvolvido em React) atua como o cliente (Single Page Application - SPA), comunicando-se com o ecossistema de microserviços através de um *API Gateway*

centralizador. O *backend*, desenvolvido em Java com Spring Boot, foi fragmentado em domínios de negócio específicos (Bounded Contexts), garantindo baixo acoplamento.

A nova topologia arquitetural é composta pelos seguintes componentes principais:

- **Identity Provider (Autenticação):** A gestão de identidades e a geração de tokens de segurança foram delegadas ao **Firebase Authentication**. O sistema não armazena credenciais confidenciais, utilizando os tokens JWT do Firebase para validar a identidade dos utilizadores (Clientes, Barbeiros e Administradores).
- **API Gateway (Spring Cloud Gateway):** Atua como a única porta de entrada pública do sistema. É responsável pelo roteamento dinâmico das requisições HTTP para os microsserviços correspondentes, resolução de CORS e interceção de segurança (validação do token do Firebase antes de permitir o acesso aos serviços internos).
- **Service Registry (Netflix Eureka):** Serviço de descoberta (*Discovery Service*) onde todos os microsserviços se registam ao iniciar. Isto permite que os serviços se comuniquem internamente através de nomes lógicos, eliminando a necessidade de *hardcoding* de endereços IP.

4.3.1 Microsserviços de Domínio

O domínio da aplicação foi subdividido em serviços independentes. Internamente, cada microsserviço mantém uma estrutura em camadas (*Controllers, Services, Repositories, Entities e Mappers*), mas macroscopicamente operam de forma isolada:

- **User Service:** Responsável pela gestão de perfis de utilizadores (Clientes e Barbeiros) e sincronização de dados de perfil com o Firebase. Ex.: Favoritar uma barbearia.
- **Barbershop Service:** Gere as entidades físicas das barbearias, convites para barbeiros (*Join Requests*) e avaliações.
- **Schedule Service:** O núcleo de negócio, responsável pela complexa gestão de disponibilidades, bloqueios de agenda e criação de agendamentos.
- **Product Service:** Módulo responsável pelo inventário, gestão de produtos físicos e controlo de fluxo de stock.
- **Payment Service:** Gere transações financeiras e integrações com *gateways* de pagamento externos (Mercado Pago), processando *webhooks*.
- **Notification Service:** Serviço isolado responsável pelo envio de comunicações e alertas (ex.: e-mails transacionais) para os utilizadores.

4.3.2 Comunicação e Persistência

A comunicação entre os serviços ocorre de duas formas distintas, otimizando a performance:

1. **Síncrona (REST/OpenFeign):** Utilizada para consultas imediatas onde um serviço necessita de dados de outro para completar uma operação (ex.: *Schedule Service* consultando os dados de um barbeiro no *User Service*).
2. **Assíncrona (Mensageria com RabbitMQ):** Adotada para operações que não exigem resposta imediata ao cliente. A aplicação utiliza o padrão *Publish-Subscribe* (Eventos). Por exemplo, ao criar um agendamento, o *Schedule Service* publica um evento `AppointmentCreatedEvent` no **RabbitMQ**, que é consumido pelo *Notification Service* para enviar um e-mail em segundo plano, não bloqueando a resposta ao utilizador.

Para a persistência de dados, a arquitetura prevê a independência de banco de dados por serviço (*Database-per-Service* pattern) utilizando **MySQL**, garantindo que uma falha num domínio não comprometa os dados de outro. Além disso, introduziu-se o uso de **Redis** como *cache* distribuído em serviços críticos para otimizar a leitura de dados frequentemente acedidos.

4.3.3 Estratégia de Notificações Ativas para Redução de *No-Show*

O absenteísmo, comumente referido no mercado como *no-show* (o não comparecimento do cliente ao horário agendado sem aviso prévio), representa um dos maiores fatores de ociosidade e de perda direta de receita no setor de estética e serviços. Para mitigar este risco e garantir a previsibilidade financeira das barbearias parceiras, o projeto CortaAí contempla a implementação de um fluxo automatizado de engajamento e notificações preventivas.

A arquitetura desta solução de comunicação foi desenhada para atuar de forma proativa nos momentos mais críticos da jornada do utilizador, sustentada pelos seguintes pilares tecnológicos e operacionais:

- **Lembretes de Proximidade (*Scheduled Tasks*):** O *Schedule Service* possui rotinas de execução em segundo plano orientadas a tempo (*cron jobs*) que monitorizam constantemente a base de dados relacional em busca de agendamentos iminentes. O sistema dispara alertas automáticos escalonados estrategicamente (como, por exemplo, 24 horas e 2 horas antes do atendimento), induzindo o cliente a confirmar a sua presença ou realizar o cancelamento com a devida antecedência.
- **Notificações *Push* via PWA:** Tirando o máximo proveito da adoção do *Progressive Web Application* (PWA), a plataforma mitiga a necessidade e os custos

associados a portais de envio de SMS. A interface utiliza as capacidades dos *Service Workers* e da *Web Push API* para despachar notificações nativas diretamente para a tela do dispositivo móvel do cliente, garantindo elevados índices de visualização e engajamento.

- **Recalibragem Dinâmica de Agenda:** Caso o cliente opte pelo cancelamento a partir da notificação recebida, o sistema executa imediatamente um fluxo de limpeza. O *Schedule Service* altera o estado da reserva, invalida a chave de disponibilidade temporária armazenada em *cache* no Redis e sinaliza a libertação daquele bloco de tempo. Isso garante que o horário vago retorne de forma instantânea para o catálogo, tornando-se elegível para novos clientes sem atrasos de sincronização.
- **Comunicação Assíncrona e Desacoplamento:** Para evitar o estrangulamento da aplicação sob picos de acesso, o processamento dos envios de notificação não concorre com as transações do sistema principal. O disparo de avisos é delegado a eventos assíncronos não bloqueantes, garantindo que as *threads* ativas que recebem novas reservas permaneçam otimizadas e com baixíssima latência.

Esta abordagem tecnológica não apenas resolve uma das principais falhas de comunicação em sistemas tradicionais, como também atua ativamente na retenção de lucro dos proprietários dos estabelecimentos, reforçando o valor de negócio agregado pelo ecossistema CortaAí.

4.3.4 Integração de Pagamentos Online e Fluxo Transacional

Para estender o valor comercial da plataforma e oferecer conveniência aos clientes, o ecossistema disponibiliza o módulo de pagamento digital integrado diretamente ao fluxo de agendamento. Esta funcionalidade é operacionalizada através do consumo da API do Mercado Pago, permitindo a liquidação instantânea de valores via Pix ou cartão de crédito.

O comportamento do sistema é regido pelos seguintes requisitos e regras operacionais:

- **Geração de Preferência de Pagamento:** No momento em que o cliente confirma a intenção de reserva no *front-end* (PWA), o microsserviço responsável intercepta a requisição e comunica-se de forma síncrona com a API do Mercado Pago para gerar uma "Preferência de Pagamento", obtendo em resposta o *Payload* contendo o *Copy and Paste* do Pix e o *QR Code* correspondente.
- **Processamento Assíncrono de Status (Webhooks):** O sistema não realiza consultas ativas (*polling*) para checar a compensação financeira. Em vez disso, expõe um *endpoint* público de *Webhook* (Notificação de IPN - *Instant Payment Notification*). Assim que o Mercado Pago processa o pagamento, ele efetua uma

chamada assíncrona para este *endpoint*, disparando a atualização automática do estado da reserva no sistema.

4.3.5 Estratégia de Engajamento e Adoção de Progressive Web Application (PWA)

No cenário competitivo de plataformas de agendamento e *marketplaces* de serviços de estética, a experiência do usuário em dispositivos móveis desempenha um papel determinante na retenção e fidelidade dos clientes.

Para mitigar essa desvantagem competitiva sem incorrer nos altos custos operacionais, na complexidade de manutenção de múltiplas bases de código distintas (Android, iOS e Web) e nas barreiras de entrada das lojas de aplicativos, o projeto adotou a estratégia de *Progressive Web Application* (PWA). Esta abordagem transforma a aplicação web principal (acessível no domínio oficial <https://web.cortaaai.shop>) em uma experiência híbrida capaz de combinar o alcance e a agilidade de atualização da web com os benefícios de engajamento de um *software* instalado.

A arquitetura de suporte ao PWA no ecossistema baseia-se em três pilares fundamentais:

- **Manifesto da Aplicação Web (*manifest.json*):** Arquivo de metadados que define o comportamento do sistema quando adicionado à tela inicial do dispositivo. Especifica os ícones em diferentes resoluções, as cores do tema e o modo de exibição *standalone*. Este modo oculta os componentes padrão do navegador (como barra de endereços e botões de navegação), garantindo uma experiência imersiva e indistinguível de um aplicativo nativo.
- **Service Workers:** Scripts executados em segundo plano pelo navegador, operando de forma independente da interface principal. Eles atuam como um *proxy* de rede, interceptando requisições e gerenciando o armazenamento local de ativos estáticos. Esse mecanismo assegura o carregamento instantâneo da interface estrutural, mesmo sob condições de conectividade intermitente ou de baixa latência.
- **Estratégia de Armazenamento do App Shell:** Em conformidade com o design distribuído do *back-end*, o PWA isola e retém localmente a casca da aplicação (componentes estruturais de navegação, estilos e roteamento). Isso garante que as requisições de rede sejam estritamente limitadas ao tráfego de dados dinâmicos transacionais gerenciados pelo *API Gateway*, reduzindo o consumo de banda e acelerando o tempo de renderização de telas críticas, como a listagem de profissionais e o fluxo de agendamento.

Dessa forma, a conversão para PWA resolve o gargalo de engajamento identificado nos concorrentes móveis dedicados, preservando a flexibilidade de integração contínua do ecossistema web e viabilizando uma distribuição imediata.

4.4 TECNOLOGIAS

Esta seção detalha a *tech stack* (pilha tecnológica) selecionada para o desenvolvimento, implantação e gestão do projeto CortaAi. A evolução para uma arquitetura de microsserviços exigiu a adoção de tecnologias robustas, voltadas para sistemas distribuídos, resiliência e alta disponibilidade.

4.4.1 Backend e Ecossistema Spring

O núcleo da aplicação servidor é construído sobre o ecossistema Java, aproveitando as ferramentas da *Spring Application Foundation*:

- **Java 17:** Linguagem de programação principal, escolhida pela sua performance, tipagem estática e suporte de longo prazo (LTS).
- **Spring Boot 3:** *Framework* base para a criação célere de microsserviços autossuficientes e com configurações nativas para a *cloud*.
- **Spring Cloud:** Conjunto de ferramentas essenciais para sistemas distribuídos, incluindo o **Spring Cloud Gateway** para o roteamento centralizado da API e o **Netflix Eureka** para o *Service Discovery* (registro e descoberta de serviços).
- **Spring Data JPA e Hibernate:** Utilizados para a abstração do mapeamento objeto-relacional (ORM) e simplificação da persistência de dados.
- **OpenFeign:** Cliente REST declarativo utilizado para a comunicação síncrona entre os microsserviços internos.

4.4.2 Mensageria e Cache

Para garantir a escalabilidade e o desacoplamento de processos pesados, o sistema integra:

- **RabbitMQ:** *Message Broker* (agente de mensagens) adotado para a comunicação assíncrona orientada a eventos, garantindo a entrega fiável de mensagens entre serviços (como disparos de e-mail e atualizações de estado).
- **Redis:** Armazenamento de estrutura de dados em memória, utilizado como *cache* distribuído para acelerar a leitura de horários disponíveis e dados de acesso frequente, reduzindo a carga no banco de dados.

4.4.3 Frontend e Autenticação

- **React com Vite:** A interface do utilizador (SPA) é desenvolvida em React, utilizando a ferramenta de *build* Vite, que proporciona um ambiente de desenvolvimento extremamente rápido e otimizado.
- **Firebase Authentication:** Serviço *Backend-as-a-Service* (BaaS) da Google utilizado como *Identity Provider*. Gere com segurança o registo, o *login* e a geração de tokens (JWT), aliviando o *backend* da gestão de credenciais sensíveis.

4.4.4 Persistência e Infraestrutura

- **MySQL:** Sistema de Gestão de Banco de Dados Relacional (SGBDR) principal, estruturado de forma a respeitar o padrão *database-per-service*.
- **Docker:** Plataforma de contentorização utilizada para empacotar cada micro-serviço com as suas dependências num ambiente isolado e padronizado, garantindo paridade entre desenvolvimento e produção.
- **Cloudinary:** Serviço de gestão de *media* em nuvem integrado para o *upload*, armazenamento e otimização de imagens (logótipos e portefólios).

4.4.5 Ferramentas de Apoio

Para a gestão do projeto e garantia de qualidade, utilizam-se as seguintes plataformas:

- **Git e GitHub:** Controlo de versões e alojamento do código-fonte, facilitando o trabalho colaborativo através de *branches* organizadas por *features*.
- **Postman / Insomnia:** Plataformas de testes de API utilizadas para modelar contratos, simular fluxos de utilizadores e validar *endpoints*.
- **Trello e Discord:** Ferramentas para a gestão de tarefas (metodologia Kanban/Scrum) e comunicação da equipe de desenvolvimento.
- **Overleaf:** Editor LaTeX colaborativo para a elaboração profissional da documentação técnica e do próprio Trabalho de Conclusão de Curso.

4.4.6 Módulo de Assistência Inteligente (Assistente Gustave)

Como diferencial estratégico focado na usabilidade e no suporte à tomada de decisão dos profissionais, o ecossistema *CortaAí* incorpora um módulo de inteligência artificial generativa, denominado *Gustave*. Este assistente atua como uma interface conversacional inteligente integrada diretamente à console administrativa do profissional (*BarberHomePage*),

viabilizando interações em linguagem natural para a extração rápida de dados e *insights* operacionais sem a necessidade de navegação manual por múltiplos menus.

O escopo funcional do assistente abrange três frentes críticas de suporte ao cotidiano da barbearia:

- **Interpretação de Dashboards e Desempenho:** O assistente é capaz de contextualizar e sumarizar os dados analíticos de faturamento, ticket médio e gráficos de produtividade apresentados na interface, traduzindo métricas visuais complexas em diagnósticos textuais diretos.
- **Consulta Dinâmica de Agendamentos:** Conectado sincronamente ao *Schedule Service*, o *Gustave* permite que o barbeiro consulte a sua fila de compromissos diários, confirme horários reservados ou resgate históricos de atendimento de determinados clientes através de simples comandos de texto.
- **Monitoramento de Itens em Estoque:** O módulo auxilia no controle operacional de insumos e produtos do estabelecimento, respondendo instantaneamente sobre níveis de inventário, alertando sobre a necessidade de reposição e prevenindo a escassez de materiais essenciais ao trabalho.

Sob a perspectiva arquitetural, o fluxo de comunicação do *Gustave* é intermediado pelo *API Gateway*, que valida rigorosamente o *token JWT* do profissional antes de delegar a requisição de chat para o microserviço ou componente responsável pela orquestração de linguagem. Essa arquitetura garante que a IA opere estritamente sob o escopo de permissões do usuário autenticado. A inclusão deste componente consolida uma infraestrutura de microserviços distribuídos capaz de sustentar de forma segura soluções de computação cognitiva voltadas ao mercado de estética.

4.4.7 Integração e Entrega Contínuas (CI/CD)

Para suportar a arquitetura de microserviços e garantir a agilidade e segurança nas entregas, o projeto adota uma estratégia robusta de Integração e Entrega Contínuas (CI/CD). Como o repositório do código-fonte é hospedado no GitHub, a ferramenta escolhida para a orquestração dos *pipelines* é o **GitHub Actions**.

A automação do ciclo de vida do *software* foi dividida em duas esteiras principais, garantindo que o código apenas chegue à infraestrutura da nuvem após validações rigorosas.

4.4.7.1 Esteira de Integração Contínua (CI)

Gatilho: Abertura ou atualização de *Pull Requests* (PRs) contra a ramificação principal (*main*).

O objetivo desta fase é garantir que o novo código é funcional, seguro e não quebra as integrações existentes. O fluxo é composto pelos seguintes passos sequenciais:

1. **Setup do Ambiente:** Provisionamento das instâncias de execução, instalação do Java 17 e configuração do *cache* de dependências do Maven para otimização do tempo de *build*.
2. **Testes Automatizados:** Execução da suíte completa de testes unitários e de integração, com o *Testcontainers* a provisionar serviços efêmeros (MySQL, Redis e RabbitMQ) de forma transparente.
3. **Análise Estática e *Quality Gate*:** Geração do relatório de qualidade e envio para o SonarQube. Caso o *Quality Gate* reprove o código (ex.: cobertura de código novo inferior a 80% ou presença de vulnerabilidades de segurança), a integração falha e o *merge* do PR é bloqueado automaticamente.

4.4.7.2 Esteira de Entrega Contínua (CD)

Gatilho: *Merge* de código previamente aprovado na ramificação principal (*main*).

Esta fase é responsável por empacotar a versão estabilizada do sistema e publicá-la no ambiente de produção. Devido à arquitetura distribuída adotada, a esteira coordena a atualização independente de cada módulo (como o *User Service*, *Schedule Service*, o *API Gateway* e a interface *Front-end* em PWA), garantindo coesão no ecossistema. O processo engloba:

1. **Geração de Artefatos:** Compilação final do código validado. No *back-end*, isso envolve o empacotamento (*.jar*) dos microsserviços Java/Spring Boot. No *front-end*, consiste no *build* otimizado dos ativos estáticos do React/Vite.
2. **Contentorização (Build & Push):** Criação das imagens Docker isoladas, restritas apenas aos serviços que sofreram alterações no ciclo atual, otimizando o tempo de processamento. Após a construção, as imagens são etiquetadas com o hash da versão e enviadas de forma segura para o registo de contentores em nuvem (Docker Hub), preparando os artefactos para implantação no servidor privado.
3. **Deploy Automatizado e Orquestração:** Atualização contínua na infraestrutura do servidor privado dedicado. O *pipeline* aciona o ambiente de produção para realizar o *pull* das novas imagens a partir do *Container Registry* (como o Docker Hub) e instanciar os contentores atualizados. A sinergia intrínseca entre o *Discovery Service* (Eureka) e o *API Gateway* garante que as novas instâncias sejam automaticamente registadas na malha de serviços e comecem a receber tráfego de

forma fluida. Simultaneamente, os contentores das versões anteriores são desregistrados do Eureka, drenados e desligados, possibilitando transições de versão altamente estáveis e minimizando a indisponibilidade para o utilizador final.

A adoção destas esteiras de CI/CD num ambiente de servidor próprio elimina o risco, a inconsistência e a morosidade das implantações manuais, fechando o ciclo de desenvolvimento com uma infraestrutura de entregas altamente automatizada, auditável e resiliente.

4.5 TESTES E MANUTENIBILIDADE

O Plano de Testes serve como um mapa ou guia para todo o esforço de teste do projeto. Em essência, ele responde às seguintes perguntas:

- **Por que** estamos testando? (Definido nos Objetivos)
- **O que** exatamente vamos testar? (Definido no Escopo de Teste)
- **Como** vamos testar? (Definido na Estratégia de Teste)
- **Com o que** vamos testar? (Definido nas Ferramentas de Teste)
- **Onde** vamos testar? (Definido nos Ambientes de Teste)
- **Quando** terminamos? (Definido nos Critérios de Conclusão)

4.5.1 Estratégia e Escopo de Teste

Foi adotado uma estratégia que segue a pirâmide de testes. O escopo de teste é definido dentro de cada nível estratégico.

4.5.1.1 Nível 1: Testes Unitários

- **Escopo e Foco:** Validação rigorosa da lógica de negócio isolada dentro de cada microsserviço. O escopo abrange primordialmente as regras de domínio implementadas nas classes de serviço (pacote `service.impl`), operando de forma independente em módulos como o *Schedule Service* e *Barbershop Service*.
- **Descrição e Ferramentas:** Cada método crítico (como o processamento lógico em `getAvailableSlots`) é testado de forma isolada. Todas as dependências externas, acessos a bancos de dados e comunicações de rede (como as chamadas via *Feign Clients* para outros microsserviços) são simuladas (*mockadas*) utilizando o *framework* Mockito em conjunto com o JUnit 5.

- **Execução:** Executados automaticamente a cada compilação local (`mvn test`) e no estágio inicial da esteira de Integração Contínua (CI), garantindo *feedback* imediato durante o desenvolvimento.

4.5.1.2 Nível 2: Testes de Integração

- **Escopo e Foco:** Validação do fluxo completo das requisições dentro das fronteiras de um único microsserviço (fluxo `Controller` → `Service` → Camada de Persistência), incluindo o mapeamento objeto-relacional e a interceptação de segurança.
- **Descrição e Ferramentas:** Utilizando `@SpringBootTest` e `MockMvc`, estes testes simulam o roteamento HTTP interno. Eles validam o funcionamento das operações transacionais de CRUD no banco de dados, a consistência de persistência (JPA/Hibernate), a rejeição adequada de cargas inválidas (`400 Bad Request` via DTOs) e a correta aplicação dos filtros de segurança, como a validação e rejeição de tokens JWT (`401 Unauthorized` / `403 Forbidden`).
- **Execução:** Realizados automaticamente durante a fase de *build* do Maven e atuam como métrica de aprovação obrigatória no *pipeline* de CI/CD, imediatamente antes da geração e envio das imagens Docker para o repositório em nuvem.

4.5.1.3 Nível 3: Testes de Componente/Integração de API

- **Escopo e Foco:** Validação dos contratos das APIs (respostas em JSON), roteamento e balanceamento correto das requisições através do *API Gateway* para os microsserviços de domínio, além da integração real com a infraestrutura de dados (MySQL e Redis) e serviços externos (Cloudinary).
- **Descrição e Ferramentas:** Após a containerização e o *deploy* no ambiente de homologação (*Staging*), são executados testes nos *endpoints* (utilizando ferramentas como Postman ou Insomnia) para verificar o fluxo completo. Isso inclui a validação dos filtros de segurança (autenticação JWT) e o comportamento transacional entre serviços distintos.
- **Execução:** Execução manual e validações exploratórias, realizadas como porta de qualidade (*Quality Gate*) rigorosa antes de qualquer promoção definitiva das imagens Docker para o ambiente de Produção.

4.5.2 Ambientes e Ferramentas de Teste

A complexidade de uma arquitetura distribuída exige que os ambientes de teste simulem fielmente a orquestração dos serviços. A tabela a seguir detalha a nova infraestrutura de

testes, agora baseada em contentores.

Tabela 5 – Ambientes e Ferramentas de Teste (Arquitetura Distribuída)

Ambiente	Propósito e Tipos de Teste	Infraestrutura e Orquestração	Ferramentas Aplicáveis
Local	Desenvolvimento e validação de lógica interna. Testes: Unitários e Integração.	Docker Compose: Orquestração local de MySQL, RabbitMQ e Redis.	JUnit 5, Mockito, Spring Boot Test, Testcontainers.
Staging	Homologação e validação de contratos entre serviços. Testes e Validação: <i>End-to-End</i> (E2E), Integração Assíncrona e Carga.	Servidor Privado Próprio executando contêineres Docker. Banco de Dados: MySQL estruturado de forma isolada por microserviço.	Postman/Insomnia, Apache JMeter e RabbitMQ Management Plugin.
Produção	Ambiente real de uso. Testes: Monitoramento de integridade (<i>Health Checks</i>).	Infraestrutura Cloud Distribuída (Docker Containers).	Firebase Console, Spring Boot Actuator.

4.5.3 Testes de Transição de Arquitetura

A transição de uma arquitetura monolítica para um ecossistema de microserviços introduz desafios significativos de qualidade, tais como a latência de rede, falhas parciais de comunicação e a validação da consistência eventual. Para mitigar estes riscos, a estratégia de testes do CortaAi foi reestruturada com base numa abordagem de camadas, garantindo a validação de cada domínio de forma isolada e integrada.

4.5.3.1 Testes Unitários (Nível de Domínio)

O primeiro nível foca-se na validação isolada da lógica de negócio contida nos *Services* e *Controllers* de cada microserviço.

- **Abordagem:** As dependências externas (bancos de dados, clientes REST e *message brokers*) são simuladas através da criação de *Mocks*.
- **Ferramentas:** **JUnit 5** como *framework* de execução e **Mockito** para a simulação de injeção de dependências.
- **Cenário Prático:** Validar se o **Schedule Service** lança corretamente uma **ConflictException** ao tentar agendar um horário que já se encontra ocupado na memória, sem necessidade de consultar o banco de dados real.

4.5.3.2 Testes de Integração e Contratos

Dada a natureza distribuída do sistema, é crucial garantir que os microsserviços compreendem os contratos uns dos outros e que interagem corretamente com as infraestruturas de persistência.

- **Integração de Base de Dados:** Utilização do **Testcontainers** para instanciar contentores Docker efêmeros (ex.: MySQL e Redis) durante a execução dos testes. Isto garante que as consultas (*queries*) JPA são validadas contra uma base de dados real idêntica à de produção.
- **Comunicação Síncrona:** Validação dos contratos de comunicação via *OpenFeign*, garantindo que o *payload* JSON esperado pelo *User Service* é corretamente enviado e interpretado pelo *Schedule Service*.

4.5.3.3 Testes de Mensageria e Eventos (Assíncronos)

Com a adoção da Arquitetura Orientada a Eventos, a estratégia de testes expandiu-se para validar a resiliência das filas de comunicação.

- **Abordagem:** Os testes de integração validam o ciclo de vida completo de uma mensagem. Verifica-se se o *publisher* formata corretamente o evento e se o *listener* o consome sem erros de desserialização.
- **Cenário Prático:** Quando um agendamento é criado, o teste valida se o evento *AppointmentCreatedEvent* é publicado na *Exchange* do **RabbitMQ** e se o *Notification Service* consegue recolher essa mensagem para gerar a intenção de envio de e-mail (validando assim a consistência eventual).

4.5.4 Ambientes e Ferramentas de Validação

Para garantir a fidelidade do ambiente de testes com a infraestrutura *Cloud*, foi estabelecida uma esteira de ambientes com orquestração local e remota, conforme detalhado na Tabela 6.

Estratégia de Integração Assíncrona: Dada a introdução do **RabbitMQ**, a estratégia de testes foi estendida para validar a consistência eventual. No ambiente de *Staging*, são realizados testes de "disparo e escuta", garantindo que um evento gerado pelo *Schedule Service* seja processado corretamente pelo *Notification Service* sem perda de mensagens em caso de falha temporária de conexão.

4.5.5 Testes funcionais

A estratégia de testes funcionais visa validar o *software* em diferentes níveis de granularidade.

Tabela 6 – Infraestrutura e Ambientes de Teste do Ecossistema

Ambiente	Propósito e Tipos de Teste	Infraestrutura e Orquestração	Ferramentas Aplicáveis
Local	Desenvolvimento e validação de lógica interna. Testes: Unitários e Integração de Repositórios.	Docker Compose: Orquestração local de MySQL, RabbitMQ e Redis.	JUnit 5, Mockito, Spring Boot Test, Testcontainers.
Staging	Homologação e validação de contratos entre serviços. Testes: E2E, Integração Assíncrona e Carga.	Instância isolada a executar imagens Docker. BD: Distribuída.	Postman / Insomnia, JMeter, RabbitMQ Admin.
Produção	Ambiente real de uso. Testes: Monitorização de integridade (<i>Health Checks</i>).	Infraestrutura Cloud Distribuída.	Firebase Console, Actuator, Logs Centralizados.

4.5.5.1 Testes Unitários

Definição: Focam em validar a menor unidade de lógica da aplicação de forma isolada (classes de Serviço). **Implementação:** Utiliza-se JUnit 5 e Mockito para simular dependências.

Cenário de Exemplo (Teste Unitário para `BarberServiceImpl.getAvailableSlots`):

Dado: Um Barber com horário das 09:00 às 18:00.

E (And): O `AppointmentsRepository` (mockado) retorna um agendamento das 10:00 às 11:00.

Quando (When): O método `getAvailableSlots` é invocado para 60 minutos.

Então (Then): O teste afirma que a lista de horários retornada contém "09:00" e "11:00", mas não contém "10:00".

4.5.5.2 Testes de Componente

Definição: Validam o comportamento de um componente ou um pequeno grupo de componentes.

- **Camada de Dados:** Utiliza-se `@DataJpaTest` do Spring Boot para validar a camada de persistência (JPA/Repositórios) contra um banco H2. Foco em consultas customizadas.
- **Contrato de API:** Utiliza-se Insomnia ou Postman para executar requisições HTTP contra a aplicação e validar os códigos de status (200, 400, 403) e a estrutura do JSON.

4.5.5.3 Testes de Integração

Definição: Pilar de validação do *backend*. Verificam a colaboração entre as camadas (Controller, Service, Repository) e a integração com o banco de dados. **Implementação:** Utiliza-se `@SpringBootTest` para carregar o contexto completo e `MockMvc` para simular requisições HTTP.

Cenário de Exemplo (POST /api/appointments):

Dado: Um banco H2 pré-populado e um usuário simulado `@WithMockUser (ROLE_CUSTOMER)`.

Quando (When): `MockMvc` executa um POST para `/api/appointments` com um DTO válido.

Então (Then): O teste afirma que a resposta HTTP é 201 (Created) e que o `AppointmentsRepository.count()` é 1.

Cenário de Exemplo (Acesso Negado):

Dado: Um usuário simulado `@WithMockUser (ROLE_CUSTOMER)`.

Quando (When): `MockMvc` executa um GET para uma rota de Dono (ex.: `/api/barbershops/my-shop/pending-requests`).

Então (Then): O teste afirma que a resposta HTTP é 403 (Forbidden).

4.5.5.4 Testes end-to-end

Definição: Simulam a jornada completa do usuário real no sistema, validando a integração entre o frontend (React) e o *backend* (Spring Boot) juntos. **Implementação:** Requer que ambas as aplicações estejam implantadas em Staging. Utiliza-se ferramentas de automação de navegador (Cypress ou Playwright).

Cenário de Exemplo (Jornada de Agendamento):

Setup: O banco de dados de teste é populado.

Login: O script abre o navegador, insere e-mail e senha de um Cliente e clica em "Entrar".

Navegação: O script clica na "Barbearia X", depois "Barbeiro Y".

Seleção: O script seleciona o "Serviço Z" e clica em um horário disponível ("14:00").

Ação: O script clica em "Confirmar Agendamento".

Verificação: O script afirma que o agendamento aparece na lista de "Meus Agendamentos".

4.5.5.5 Cobertura de Cenários Negativos e Casos de Borda

Uma estratégia de testes de *software* robusta não se limita a validar o "caminho feliz" (*happy path*). Em ecossistemas distribuídos, a garantia de qualidade é comprovada pela resiliência do sistema perante entradas inválidas, violações de integridade estrutural e falhas de concorrência. Para assegurar esta fiabilidade, foram mapeados e automatizados diversos cenários negativos nas camadas de unidade e integração.

4.5.5.5.1 Cenários de Validação de Domínio (Testes Unitários)

Estes testes focam-se na rejeição de estados inválidos pela lógica de negócio interna, operando em memória sem dependência do banco de dados.

- **Agendamento fora do expediente da barbearia:**

Dado um Barbeiro com o horário de trabalho configurado das 09:00 às 18:00.

Quando um Cliente tenta agendar um serviço com início às 18:30.

Então o *Schedule Service* lança uma *BusinessValidationException* com a mensagem "Horário selecionado encontra-se fora do expediente do profissional".

- **Incompatibilidade de Serviço (*Skills*):**

Dado um Barbeiro que possui apenas a atividade "Corte de Cabelo" vinculada ao seu perfil.

Quando o Cliente tenta agendar "Barba e Sobrancelha" com este profissional.

Então a lógica de serviço bloqueia a ação, Impedindo que o mesmo prossiga.

- **Barbeiro sem jornada de trabalho definida:**

Dado um Barbeiro recém-aprovado numa barbearia que ainda não configurou o seu horário de expediente (*work_start_time* nulo).

Quando o Cliente tenta consultar a disponibilidade diária deste profissional.

Então o sistema não gera exceções não tratadas (*NullPointerException*), mas retorna graciosamente uma lista vazia, impedindo qualquer alocação.

4.5.5.5.2 Cenários de Restrição e Concorrência (Testes de Integração)

Estes testes asseguram que a camada de persistência e os controladores REST lidam adequadamente com violações no modelo relacional e acessos simultâneos.

- **Violação de Dados Únicos (Duplicidade de CPF/E-mail):**

Dado que o CPF "123.456.789-00" já se encontra registado e ativo no *User Service*.

Quando um novo utilizador tenta concluir o seu registo com o mesmo CPF.

Então o repositório lança uma *DataIntegrityViolationException*. O *GlobalExceptionHandler* da aplicação interceta esta falha nativa do banco de dados e

converte-a num *status* HTTP 409 (*Conflict*), devolvendo uma mensagem legível ao cliente sem expor a estrutura interna do SQL.

- **Concorrência de Agendamento (*Race Condition*):**

Dado que existe apenas um bloco de horário disponível às 14:00 para um determinado barbeiro.

Quando dois Clientes distintos tentam reservar exatamente o mesmo horário ao mesmo tempo (no mesmo milissegundo).

Então os mecanismos de isolamento transacional (*Locks* de banco de dados) garantem que apenas a primeira requisição seja confirmada com HTTP 201 (*Created*), enquanto a segunda requisição é rejeitada com HTTP 409 (*Conflict*), impedindo a ocorrência de *overbooking*.

- **Falha de Autorização Horizontal (Insegurança Direta de Objeto):**

Dado um Cliente autenticado com o UID "A" através do Firebase.

Quando este tenta aceder ao *endpoint* de cancelamento passando o ID de um agendamento pertencente ao Cliente "B".

Então a camada de segurança (*Controller* / *Service*) deteta que o proprietário do *token* não tem direitos sobre o recurso solicitado e retorna o código HTTP 403 (*Forbidden*).

4.5.6 Testes não funcionais

Verificam como o sistema opera sob determinadas condições, focando em desempenho, confiabilidade e configuração.

4.5.6.1 Testes de Performance

Definição: Medem a velocidade, responsividade, taxa de transferência (*throughput*) e estabilidade do ecossistema distribuído sob carga.

- **Testes de Estresse (*Stress Testing*):** Aumenta progressivamente o volume de requisições além da capacidade esperada para identificar o ponto de ruptura da infraestrutura. O foco é avaliar a resiliência do *API Gateway* como ponto único de entrada, o esgotamento do *pool* de conexões no banco de dados relacional (MySQL) e no Redis, além dos limites de consumo de CPU e memória dos contentores Docker.
- **Testes de Latência e Eficácia de Cache:** Como contêineres possuem execução contínua (eliminando a latência de *Cold Start* inerente ao modelo *serverless*), o monitoramento de performance passa a focar no tempo de resposta das comunicações síncronas na malha de serviços. Mede-se, por exemplo, a eficiência da resposta do

sistema ao processar o *endpoint* de disponibilidade de agendas, validando empiricamente a redução de carga no banco de dados proporcionada pela estratégia de *caching* do Redis em cenários de alta concorrência.

4.5.6.2 Testes de Configuração

Definição: Validam que o ecossistema de microsserviços funciona corretamente e de forma coesa em seu ambiente de implantação final containerizado.

- **Validação de Variáveis de Ambiente:** Verifica se os contêineres conseguem ler e aplicar as variáveis essenciais injetadas durante o *deploy* (ex.: `SPRING_DATASOURCE_URL`, `SPRING_DATA_REDIS_HOST`, `EUREKA_CLIENT_SERVICEURL_DEFAULTZONE`, `CLOUDINARY_URL` e a chave secreta do JWT).
- **Validação de Conectividade de Rede:** Garante que as políticas de rede (como as *Docker networks* ou *Security Groups* na nuvem) permitam a descoberta de serviços via Eureka, o roteamento correto a partir do *API Gateway*, e o acesso ininterrupto aos bancos de dados (MySQL e Redis) e integrações externas (Cloudinary).
- **Validação de Configuração da Aplicação:** Testa se as parametrizações específicas definidas nos arquivos `application.yml` de cada serviço (ex.: `app.availability.slot-interval-minutes:15` isolado no *Schedule Service*) estão sendo carregadas corretamente na inicialização da aplicação.
- **Validação de Configuração de Segurança:** Testa se as regras definidas no `SecurityConfig` e a injeção do filtro `JwtAuthorizationFilter` estão operando ativamente em cada domínio, garantindo que o acesso a rotas protegidas seja validado e rejeitado quando necessário, assegurando o isolamento das regras de autorização de cada microsserviço.

4.5.7 Testes automatizados

Para a geração de arquivos de logs no *backend* (Java/Spring Boot), são utilizadas as ferramentas padrão SLF4J (fachada) e *Logback* (implementação). Os níveis de log utilizados seguem a semântica padrão:

- **ERROR:** Falhas críticas (ex.: falha ao conectar no banco).
- **WARN:** Eventos inesperados (ex.: tentativa de cancelar agendamento já cancelado).
- **INFO:** Mensagens informativas de progresso (ex.: confirmação de criação/atualização).

- **DEBUG:** Mensagens detalhadas para desenvolvimento (ex.: visualização de queries SQL).
- **TRACE:** Nível mais detalhado (ex.: valores de parâmetros de queries).

4.5.8 Análise estática

Visto a importância de se preservar a qualidade e segurança do código, é essencial realizar a Análise Estática de Código e Segurança (SAST). Para isso, empregaremos a ferramenta SonarQube. O SonarQube é uma plataforma de código aberto para inspeção contínua da qualidade do código, integrada ao processo de build (Maven). Ele analisa o código-fonte Java em busca de:

- Bugs: Erros potenciais na lógica.
- Vulnerabilidades de Segurança: Falhas que podem ser exploradas.
- Code Smells: Problemas de design ou manutenibilidade.
- Duplicação de Código: Trechos de código repetidos.
- Cobertura de Testes: Verifica a porcentagem do código coberta pelos testes unitários (JUnit).

4.5.9 Code convention

Para garantir a qualidade, legibilidade e manutenibilidade do código-fonte, foram adotadas convenções de codificação padrão.

4.5.9.1 Convenções de Backend (Java / Spring Boot)

A convenção de código adotada é o **Google Java Style Guide**, complementado por práticas comuns do Spring Boot.

- **Nomenclatura:**
 - Classes e Interfaces: *UpperCamelCase*.
 - Métodos e Variáveis: *lowerCamelCase*.
 - Constantes: `CONSTANT_CASE`.
 - Pacotes: *lowercase* (ex.: `ifsp.edu.projeto.cortaaai.service.impl`).
- **Formatação:** Indentação de 4 espaços e organização padrão de `import`.
- **Práticas:** Uso extensivo de anotações do Spring (`@Service`, `@RestController`) e JPA (`@Entity`, `@Id`), com comentários no padrão Javadoc.

4.5.9.2 Convenções de Frontend (React)

A convenção a ser adotada será o **Airbnb JavaScript Style Guide**, incluindo suas extensões para React.

- **Nomenclatura:**

- Componentes React: *UpperCamelCase* (ex.: `AppointmentForm`).
- Variáveis, Funções e Props: *lowerCamelCase*.

- **Práticas:** Preferência por componentes funcionais com *Hooks*, uso de `const` sobre `let`, e **arrow functions**.
- **Formatação:** A consistência do estilo será garantida automaticamente por ferramentas como **Prettier** e **ESLint**.

4.6 SEGURANÇA E CONTROLE DE ACESSO

Numa arquitetura de microsserviços, a segurança não pode ser gerida de forma isolada por cada serviço, sob pena de criar vulnerabilidades e redundância de código. Por isso, o CortaAi adotou um modelo de **Identidade Federada e Roteamento Seguro**, removendo a responsabilidade de gestão de credenciais (senhas) do banco de dados interno.

A estratégia de segurança divide-se em três pilares principais: Provedor de Identidade, Interceção no *Gateway* e Autorização Baseada em Funções (RBAC).

4.6.1 Provedor de Identidade (Firebase Authentication)

O sistema utiliza o **Firebase Authentication** (Google) como provedor de identidade (*Identity Provider* - IdP). O *backend* do CortaAi **não armazena senhas** em nenhum dos seus bancos de dados (MySQL).

O fluxo de autenticação ocorre da seguinte forma:

1. O cliente (Frontend em React) envia as credenciais (*e-mail* e senha) diretamente para os servidores seguros do Firebase.
2. Após a validação, o Firebase retorna um *JSON Web Token* (JWT) assinado assimetricamente ao cliente, contendo o *User ID* (UID) único daquele utilizador.
3. Este token passa a ser enviado pelo Frontend no cabeçalho (*Header*) **Authorization: Bearer <token>** em todas as requisições subsequentes para a API.

Esta abordagem garante conformidade com as melhores práticas de segurança de mercado, transferindo a responsabilidade de encriptação, recuperação de senhas e proteção contra ataques de força bruta para uma infraestrutura globalmente testada.

4.6.2 Ciclo de Vida do Token e Estratégia de Renovação

Diferente de sistemas tradicionais que utilizam sessões mantidas em memória no servidor (*stateful*), o ecossistema adota uma abordagem estritamente **Stateless**. Isto significa que os microsserviços não armazenam o estado de sessão do utilizador; cada requisição é autossuficiente, contendo um *token* digital assinado criptograficamente que trafega de forma segura entre os componentes da arquitetura.

O ciclo de vida das credenciais é dividido em dois componentes principais fornecidos pelo Firebase:

- **ID Token (Access Token):** É o JWT emitido pelo Firebase Auth no momento do *login*, possuindo uma curta duração com validade fixa de **60 minutos**. Este *token* é anexado ao cabeçalho de cada chamada HTTP direcionada à API. Devido ao seu curto tempo de vida, a superfície de exposição em caso de interceptação maliciosa é severamente minimizada.
- **Refresh Token:** É um *token* de longa duração persistido de forma segura pelo *front-end* cliente. Este componente não é enviado para as rotas operacionais dos microsserviços de negócio, sendo utilizado de forma exclusiva em segundo plano para interagir com os servidores de autorização do Firebase, automatizando a renovação transparente do *ID Token* quando este expira.

O fluxo de renovação (*Silent Refresh*) ocorre inteiramente no lado do cliente. Através de interceptores no Frontend (utilizando o SDK do Firebase), o sistema verifica a validade do *ID Token* antes de cada requisição. Caso o token esteja prestes a expirar, o cliente utiliza o *Refresh Token* para solicitar um novo *ID Token* de forma transparente, garantindo que o utilizador permaneça autenticado sem interrupções na experiência de uso, enquanto o *backend* mantém-se protegido e livre de gestão de sessões pesadas.

4.6.3 Interceção e Proteção (API Gateway)

O **Spring Cloud Gateway** atua como o único ponto de entrada público do ecossistema e o principal guardião de segurança.

Foi implementado um filtro de interceção customizado (`FirebaseTokenGatewayFilter`) que analisa todas as requisições de entrada. O seu papel é:

- **Validação da Assinatura:** O *Gateway* verifica a validade criptográfica do JWT contra as chaves públicas da Google. Se o token for inválido, expirado ou ausente, a requisição é bloqueada imediatamente com o erro **401 Unauthorized**, protegendo os microsserviços internos de tráfego malicioso.
- **Propagação de Contexto:** Após validar o token, o *Gateway* extrai o UID do utilizador e injeta-o como um novo cabeçalho HTTP interno (ex.: **X-User-Id**). Desta

forma, microsserviços como o *Schedule Service* ou *User Service* podem identificar quem está a fazer a requisição sem precisarem de revalidar o token criptográfico ou de importar as bibliotecas do Firebase.

4.6.4 Política de Senhas Robustas e Credenciais

A integridade das contas de utilizadores é garantida através de uma política de senhas rigorosa, aplicada diretamente no momento do registo e alteração de credenciais. Uma vez que a autenticação é delegada ao Firebase, o sistema beneficia de mecanismos de proteção contra ataques de dicionário e *brute force* nativos da infraestrutura da Google.

As regras de complexidade configuradas para o ecossistema CortaAi exigem:

- **Comprimento Mínimo:** Todas as senhas devem conter, no mínimo, 6 caracteres (padrão de segurança do Firebase).
- **Complexidade Alfanumérica:** Recomenda-se a combinação de letras maiúsculas, minúsculas, números e caracteres especiais, validada através de expressões regulares (*regex*) no Frontend antes do envio ao provedor.
- **Proteção contra Reutilização:** O sistema impede o uso de senhas excessivamente simples ou sequenciais.

Além da validação de formato, o Firebase Authentication utiliza algoritmos de *hashing* avançados (uma versão otimizada do **scrypt**) para armazenar as credenciais de forma irreversível. Isto garante que, mesmo em cenários hipotéticos de acesso não autorizado aos registos do provedor, as senhas originais dos utilizadores permanecem protegidas. No fluxo de alteração de senha, o sistema exige a reautenticação do utilizador ou a validação através de um *link* seguro enviado para o *e-mail* verificado, garantindo que apenas o proprietário legítimo possa modificar as suas credenciais.

4.6.5 Autorização Baseada em Funções (RBAC)

Enquanto o Firebase resolve o problema da *Autenticação* (quem é o utilizador), o sistema interno resolve o problema da *Autorização* (o que ele pode fazer).

O *User Service* mantém o mapeamento entre o UID do Firebase e a função (*Role*) do utilizador dentro da plataforma. Foram definidas três funções principais protegidas pelo *Spring Security* interno de cada serviço:

- **ROLE_CUSTOMER:** Acesso aos próprios agendamentos e visualização de barbearias públicas.
- **ROLE_BARBER:** Acesso à gestão da própria agenda, bloqueios de horário e visualização de comissionamentos.

- **ROLE_OWNER:** Privilégios administrativos, como a gestão de inventário (*Product Service*), aprovação de barbeiros na barbearia e métricas financeiras globais (*Payment Service*).

4.6.6 Observância à Legislação

O projeto foi desenvolvido com atenção à conformidade com a Lei Geral de Proteção de Dados (LGPD - Lei nº 13.709/2018):

- **Finalidade Específica e Consentimento:** A LGPD exige que dados pessoais (nome, telefone, e-mail, CPF) sejam coletados com consentimento explícito e utilizados apenas para a finalidade informada (agendamentos, gerenciamento).
- **Proteção de Dados Pessoais:** Medidas como criptografia de senhas, controle de acesso e validação de dados são implementadas para proteger os dados.
- **Direitos dos Titulares:** A LGPD garante aos titulares (usuários) direitos como acesso, correção, eliminação e portabilidade de seus dados.
- **Transparência:** A política de privacidade da aplicação deve informar claramente quais dados são coletados, como são utilizados e por quanto tempo são armazenados.

4.7 MODELO DE BANCO DE DADOS

Para a persistência de dados, o projeto adota o padrão *Database-per-Service* em conformidade com a arquitetura de microsserviços. Esta abordagem garante que cada domínio (como *User*, *Barbershop* e *Schedule*) possua a sua própria base de dados isolada (MySQL). Este design assegura o baixo acoplamento, permitindo escalabilidade independente e garantindo que manutenções ou falhas pontuais num serviço não comprometam a disponibilidade global do ecossistema. Adicionalmente, foi implementada uma estratégia transversal de chaves primárias baseadas em UUID (Universally Unique Identifier) e exclusão lógica (*Soft Delete*) para mitigar vulnerabilidades (como ataques IDOR) e preservar o histórico financeiro e de auditoria.

4.7.1 Modelo entidade relacionamento - MER

O Modelo Entidade-Relacionamento (MER) descreve as entidades conceituais, as suas interações e as regras de negócio intrínsecas, abstraindo a separação física. Embora a implementação final seja distribuída, o MER global consolida os seguintes domínios:

- **Domínio de Utilizador (*User Service*):** Entidades responsáveis por armazenar os perfis, credenciais, competências e dados de contato de clientes e barbeiros.

- **Domínio de Barbearia (*Barbershop Service*):** Gere as informações cadastrais dos estabelecimentos, a identidade visual (portfólio via Cloudinary), o catálogo de serviços dinâmico e o fluxo de requisições para a formação de equipas.
- **Domínio de Agendamento (*Schedule Service*):** Motor transacional que orquestra a relação entre o cliente, o barbeiro, a barbearia, os serviços selecionados e os blocos de horário reservados.

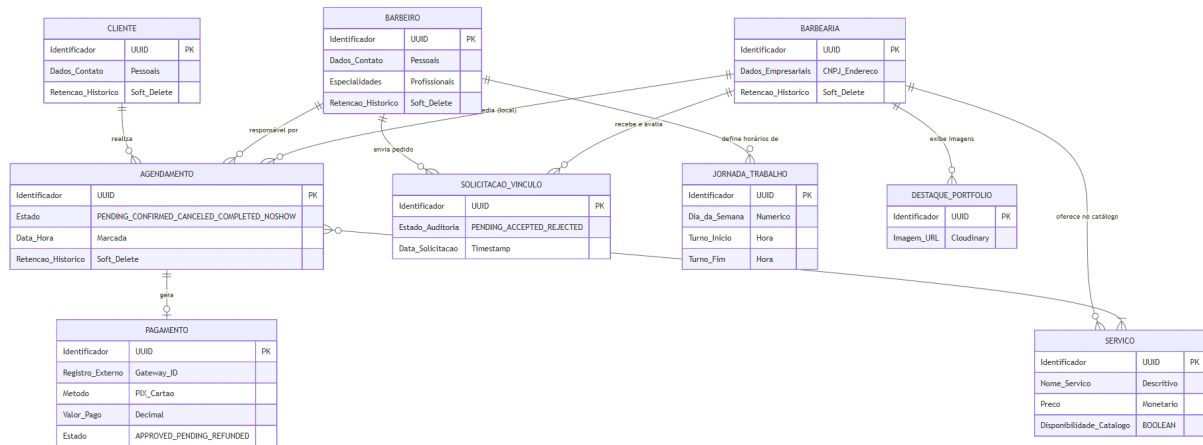


Figura 3 – Modelo Entidade-Relacionamento (MER) - Foco em Domínios e Regras de Negócio

4.7.2 Diagrama entidade relacionamento - DER

O Diagrama Entidade-Relacionamento (DER) detalha a estrutura física das tabelas, a tipagem rigorosa dos dados e o isolamento dos microsserviços. É fundamental notar que, neste paradigma distribuído, as relações entre entidades de serviços diferentes (ex.: um **Appointment** no *Schedule Service* vinculado a um **Customer** no *User Service*) são mantidas exclusivamente através de **Referências Lógicas** (UUIDs partilhados) e consistência eventual, substituindo as tradicionais chaves estrangeiras (*Foreign Keys*) físicas transversais.

Isto garante que o serviço de agendamentos orquestre reservas sem a necessidade de aceder diretamente ou bloquear tabelas do serviço de utilizadores, comunicando-se unicamente via *API Gateway* ou mensageria.

As principais entidades (tabelas) que compõem o sistema físico distribuem-se da seguinte forma:

BARBERSHOPS (Barbearias): Contém os dados cadastrais da empresa (nome, CNPJ, endereço e *logo_url*). Protegida por exclusão lógica (*deleted_at*).

CUSTOMERS (Clientes): Armazena as informações dos clientes finais. Adota UUID primário e controle de *Soft Delete* para retenção histórica.

BARBERS (Barbeiros): Contém os dados profissionais (nome, telefone, CPF, e competências em *skills*). Inclui mapeamento para a gestão da sua própria jornada de trabalho (*BARBER_WORK_HOURS*).

ACTIVITIES (Atividades/Serviços): Representa os procedimentos oferecidos (nome, preço, duração). Incorpora a *flag is_active*, permitindo desativar um serviço do catálogo público sem corromper o histórico financeiro de agendamentos passados.

APPOINTMENTS (Agendamentos): Entidade central transacional. Utiliza UUID na chave primária para ofuscar o volume de reservas (prevenção contra vazamento de inteligência de negócio) e registra o estado da marcação, além das referências lógicas para cliente, barbeiro e barbearia.

BARBERSHOP_JOIN_REQUESTS: Tabela de auditoria para gestão de equipes. Regista o ciclo de vida completo do vínculo profissional, transitando entre os estados de PENDING, ACCEPTED e REJECTED, marcados por *timestamps* de atualização para garantir conformidade legal.

PAYMENTS (Pagamentos): Entidade transacional do *Schedule Service* dedicada ao isolamento do fluxo financeiro. Armazena o identificador de referência externa gerado pela API do Mercado Pago (*external_payment_id*), o método de liquidação, o valor processado e o estado da transação (ex: APPROVED, REFUNDED). A sua separação da entidade de agendamentos garante a integridade dos dados caso ocorram múltiplas tentativas de pagamento ou falhas no *checkout*.

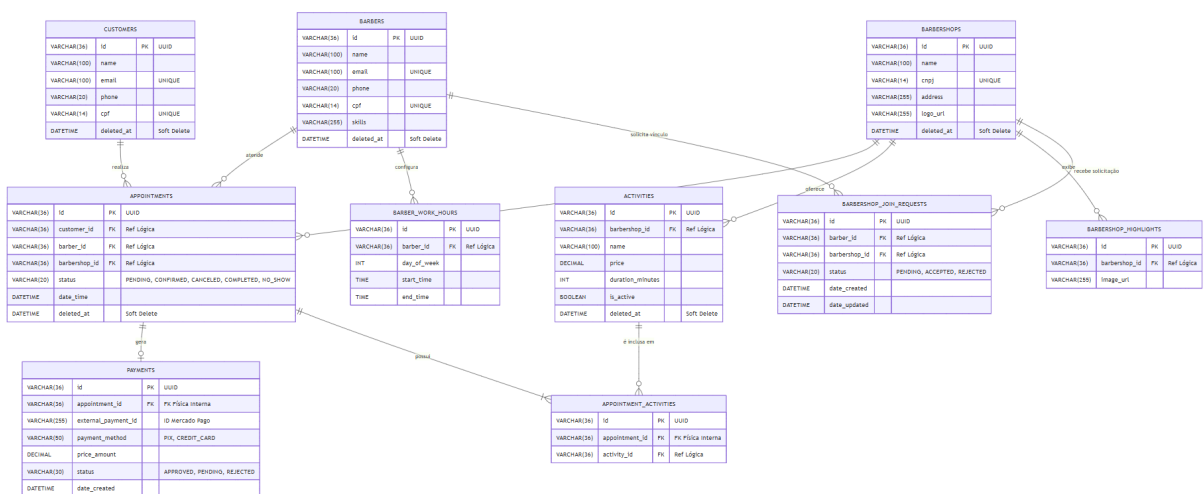


Figura 4 – Diagrama Entidade-Relacionamento (DER) - Modelo Lógico/Físico Descentralizado

4.7.2.1 Tabelas de Junção (M:N)

Em arquiteturas de microsserviços, a normalização por junção ocorre estritamente dentro das fronteiras de um mesmo domínio:

APPOINTMENT_ACTIVITIES: Especifica quais atividades (`activity_id`) estão incluídas num determinado agendamento (`appointment_id`). Esta tabela resolve a cardinalidade de serviços compostos (ex.: "Corte + Barba" na mesma reserva).

4.7.2.2 Relacionamentos Chave (Cardinalidade)

- **Barbearia e Barbeiro (Emprega / Vincula):** Uma barbearia gere uma equipa de múltiplos barbeiros (1:N). Este vínculo é formalizado logicamente e auditado através do fluxo de `BARBERSHOP_JOIN_REQUESTS`.
- **Barbearia e Atividade (Oferece):** Uma barbearia (`BARBERSHOPS`) oferece múltiplas atividades (`ACTIVITIES`) no seu catálogo dinâmico (1:N).
- **Cliente e Agendamento (Realiza):** Um cliente (`CUSTOMERS`) realiza e detém um histórico de vários agendamentos ao longo do tempo (1:N).
- **Barbeiro e Agendamento (Atende):** Um profissional (`BARBERS`) é o prestador responsável por múltiplos agendamentos (`APPOINTMENTS`) na sua jornada (1:N).
- **Agendamento e Atividade (Inclui):** O relacionamento entre `APPOINTMENTS` e `ACTIVITIES` é de muitos para muitos (M:N), resolvido internamente na base de dados do *Schedule Service* pela tabela física `APPOINTMENT_ACTIVITIES`.
- **Agendamento e Pagamento (Liquida / Orquestra):** Um agendamento (`APPOINTMENTS`) pode gerar uma ou mais transações financeiras (`PAYMENTS`) (1:N, considerando cenários de falha na primeira tentativa e reprocessamento). O isolamento deste relacionamento evita a sobrecarga da entidade de reservas, permitindo que os *Webhooks* do Mercado Pago atualizem exclusivamente os registos financeiros até à confirmação final do serviço.

4.8 CRONOGRAMA DO PROJETO

A execução do projeto seguirá o framework ágil Scrum. O cronograma não é baseado em fases sequenciais, mas em uma série de ciclos iterativos (Sprints).

- **Cadência das Sprints:** 8 Sprints de 2 semanas (10 dias úteis) cada.
- **Product Backlog:**
 - Épico 1: Gestão de Acesso (Autenticação JWT, Perfis).

- Épico 2: Módulo de Agendamento (Visualização, criação/cancelamento).
- Épico 3: Módulo de Serviços e Inventário (Cadastro, preços, duração).
- Épico 4: Painel Administrativo (Gestão de agenda e serviços).
- Épico 5: Infraestrutura e Implantação (Configuração, RDS, CI/CD).

4.8.1 Cronograma de Sprints - Fase Inicial (2025)

A Tabela 8 apresenta o planejamento executado durante a fase inicial de desenvolvimento do projeto, focada na construção e fundação do sistema.

Tabela 7 – Cronograma de Sprints - Desenvolvimento Inicial (2025)

Sprint	Período	Meta da Sprint (Sprint Goal)	Marcos Equivalen-tes
Sprint 1	15/08 a 29/08	Setup e Fundação: Configurar ambientes (Git, CI/CD, Trello), definir o Product Backlog (MVP) e criar o <i>Walking Skeleton</i> .	M1: Requisitos e Es-copo Aprovados
Sprint 2	01/09 a 12/09	Autenticação: Implementar o fluxo de autenticação (JWT) e o CRUD básico de Usuários. Definir a arquitetura de base.	M2: Arquitetura e De-sign Concluídos
Sprint 3	15/09 a 26/09	MVP do Agendamento (Back-end): API para lógica de horários disponíveis e criação de agendamento.	-
Sprint 4	29/09 a 10/10	MVP de Serviços (Back-end): APIs para CRUD de Serviços e Inventário.	M3: Desenvolvimento Back-end Concluído
Sprint 5	13/10 a 24/10	MVP do Cliente (Front-end): Telas de Login e fluxo principal do cliente (ver horários e realizar agendamento).	-
Sprint 6	27/10 a 07/11	Painel do Barbeiro (Front-end): Telas para o barbeiro visualizar e gerenciar sua agenda e serviços.	M4: Desenvolvimento Front-end Concluído
Sprint 7	10/11 a 21/11	Testes de Aceite (UAT): Foco em testes de integração (E2E), correção de bugs e validação do fluxo com usuários-piloto.	M5: Ciclo de Testes (UAT) Finalizado
Sprint 8	24/11 a 05/12	Hardening e Implantação: Preparação do ambiente de produção (AWS), testes, deploy e documentação final.	M6: Implantação (Go-Live)

4.8.2 Cronograma do projeto (Fase de Evolução e Estabilização)

A gestão do projeto foi adaptada para um ciclo focado na refatoração arquitetural para microserviços e no polimento do Produto Mínimo Viável (MVP). O cronograma a seguir compreende o período crítico iniciado em fevereiro de 2026, estendendo-se até a homologação final em maio do mesmo ano.

Tabela 8 – Cronograma do projeto (Fase de Evolução e Estabilização)

Sprint	Período (Início/Fim)	Meta da Sprint (Sprint Goal)	Marcos Equivalen-tes (Novo Plano)
Sprint 1	27/02 13/03	Desacoplamento: Fragmentação do monólito em serviços independentes e configuração do API Gateway.	M7: Arquitetura Distribuída Inicial
Sprint 2	16/03 27/03	Infraestrutura: Implementação do Netflix Eureka para descoberta de serviços e configuração do ambiente Docker.	M8: Orquestração de Containers
Sprint 3	30/03 10/04	Segurança e Identidade: Migração total para Firebase Authentication e validação de permissões por Roles.	M9: Provedor de Identidade Migrado
Sprint 4	13/04 24/04	Estabilização de Dados: Sincronização entre serviços via OpenFeign e ajustes de persistência no cache Redis.	-
Sprint 5	27/04 08/05	Mensageria: Implementação do sistema de notificações consumindo eventos do RabbitMQ.	M10: Sistema de Eventos Concluído
Sprint 6	11/05 22/05	QA e Bug Fixing: Ciclo intensivo de correção de bugs, testes de carga nos micro-serviços e refinamento da interface.	M11: Homologação Técnica
Sprint 7	25/05 27/05	Homologação Final: Preparação do ambiente de produção (Cloud), documentação final e encerramento do projeto.	M12: Entrega Formal do TCC

4.8.3 Análise do Caminho Crítico e Riscos

Nesta fase final, o caminho crítico desloca-se do desenvolvimento de novas funcionalidades para a resiliência da infraestrutura distribuída. Os principais pontos de atenção são:

- **Dependência de Integração:** O principal risco reside na integração do sistema de notificações via *RabbitMQ*. Falhas na publicação ou consumo de eventos podem gerar inconsistências na comunicação com o usuário final.
- **Estabilização de Ambiente:** O período de *Bug Fixing* (Sprint 6) é vital para garantir que a orquestração dos contêineres Docker mantenha a performance sob carga real de uso.
- **Restrição Temporal:** A data de 27 de maio de 2026 é uma restrição rígida, exigindo que o polimento do sistema não comprometa o funcionamento dos requisitos básicos de agendamento e pagamento.
- **Reserva Explícita de Tempo para Infraestrutura (DevOps):** Para evitar o acúmulo de débito técnico na infraestrutura e garantir a validade dos testes de Integração e *End-to-End* (E2E), este cronograma estabelece uma reserva fixa de esforço: **20% do tempo útil de cada Sprint** (o equivalente a 2 dias úteis num ciclo quinzenal) é dedicado exclusivamente a tarefas de DevOps e manutenção de ambiente.

4.9 PROTOTIPAGEM E DESIGN DE INTERFACE (UI/UX)

Para garantir que a plataforma CortaAí entregasse uma experiência de uso intuitiva e alinhada às expectativas dos seus diferentes perfis de utilizadores, a concepção do *front-end* foi precedida por uma etapa rigorosa de prototipagem e design de interface.

A etapa de prototipagem de alta fidelidade permitiu validar os fluxos de navegação antes da codificação em React. Com base nas telas desenvolvidas e homologadas, destacam-se os seguintes fluxos principais:

- **Autenticação e Seleção de Papéis:** O fluxo de entrada apresenta de forma clara a proposta de valor ("Seu estilo, sua hora marcada") e exige que o utilizador defina o seu contexto de uso através das opções "Entrar como Cliente" ou "Entrar como Barbeiro". Os formulários de *login* e criação de conta foram desenhados para exigir apenas os dados estritamente necessários, reduzindo o atrito inicial.
- **Descoberta e Busca (*Home Page*):** Tela principal do utilizador cliente, equipada com uma barra de pesquisa superior. O *layout* divide-se em um carrossel de rolagem horizontal para "Minhas Barbearias Favoritas" e uma listagem vertical detalhada de "Barbearias Disponíveis", exibindo a imagem do estabelecimento, avaliação por estrelas, nome e distância.
- **Fluxo de Agendamento:** A tela de conversão principal estrutura a reserva em uma única página rolável de fácil compreensão. O utilizador seleciona categorias de serviços através de *chips* visuais (ex: Corte, Barba), escolhe o profissional através de cartões com foto e *status* de disponibilidade, e finaliza selecionando a data e o bloco de horário em um componente de calendário simplificado.
- **Gestão de Perfil e Histórico:** A interface "Meu Perfil" atua como o centro de informações do utilizador. Além de consolidar os dados de contacto, exhibe de forma clara a secção "Meus Agendamentos", que funciona como um extrato de serviços, listando o procedimento realizado, o profissional responsável, a data, o horário e o valor pago (R\$).

As Figuras a seguir apresentam os protótipos conceituais que serviram de base arquitetural para a construção e componentização visual no *front-end*.



Figura 5 – Protótipo de Alta Fidelidade - Fluxo de Entrada, Seleção de Perfil e Cadastro

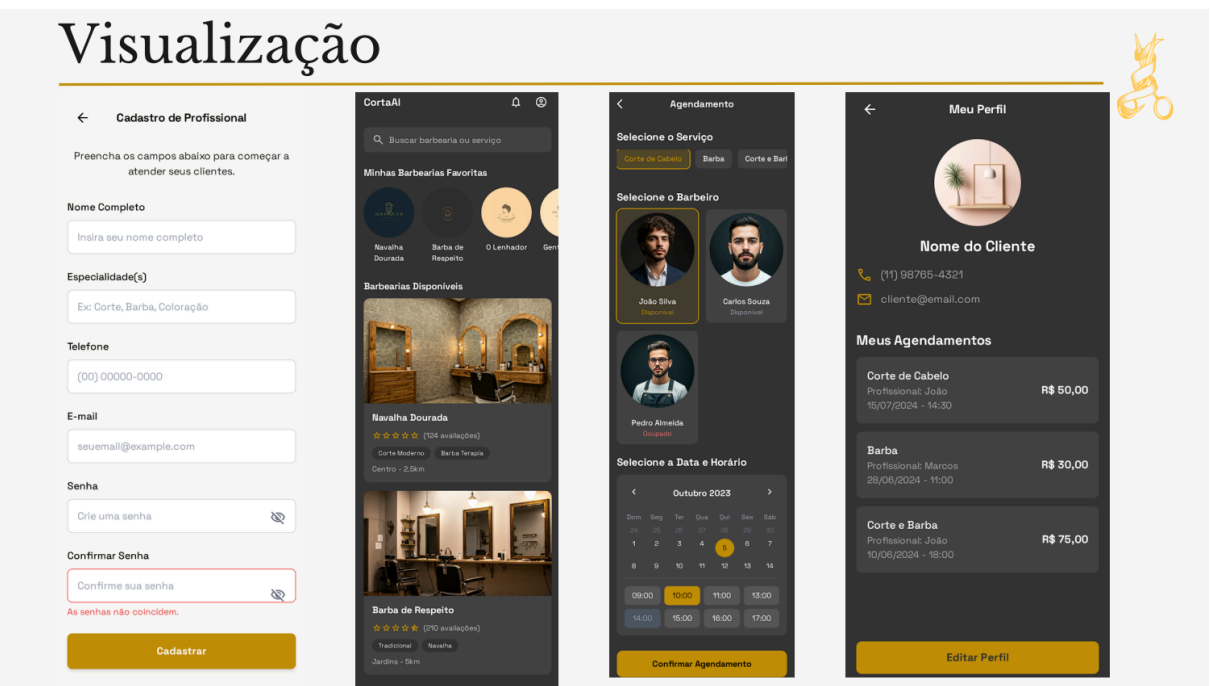


Figura 6 – Protótipo de Alta Fidelidade - Descoberta de Barbearias, Fluxo de Agendamento e Perfil

5 CONSIDERAÇÕES FINAIS

O desenvolvimento da aplicação web "CortaAí", um *marketplace* para gestão e agendamento em barbearias, representa uma iniciativa significativa para modernizar e otimizar a operação destes estabelecimentos, trazendo organização, eficiência e maior visibilidade no ambiente digital. Ao evoluir para uma arquitetura distribuída e escalável baseada em microsserviços (ecossistema Java 17 e Spring Boot containerizado via Docker, orquestrado com *API Gateway* e *Service Discovery*), em conjunto com uma interface baseada em *Progressive Web Application* (PWA) em React, foi possível estruturar uma solução robusta e resiliente que atende perfeitamente às complexidades de concorrência de dados e à gestão de múltiplos perfis e estabelecimentos.

Ao longo do projeto, foi fundamental analisar e implementar funcionalidades que atendessem de forma coesa às necessidades dos diferentes atores do sistema: Clientes, buscando conveniência móvel para encontrar barbearias e agendar serviços online; Barbeiros, necessitando de uma ferramenta para gerenciar o seu perfil, habilidades, jornadas de trabalho e agenda; e Donos de barbearias, visando o controle sobre o fluxo de *onboarding* da loja, catálogo de serviços, gestão da equipe e aprovação de vínculos. A implementação da autenticação *stateless* via JWT, combinando a filtragem de borda no *API Gateway* com a validação descentralizada na malha de serviços, garante uma segurança rigorosa e o acesso isolado às funcionalidades adequadas de cada perfil.

A plataforma "CortaAí" não só permite aos Clientes realizar agendamentos de forma eficiente, verificando a disponibilidade em tempo real — operação agora otimizada por uma camada de *cache* em memória (Redis) que reduz drasticamente a latência e a sobrecarga no banco de dados relacional (MySQL) —, mas também oferece aos profissionais ferramentas intuitivas para o controle do seu negócio. A gestão avançada de portfólio e identidade visual (via integração com o *Cloudinary*) e a facilidade de acesso em tela cheia proporcionada pelo PWA elevam o engajamento e aproximam a experiência web à de um aplicativo nativo.

Além disso, a modularidade e o isolamento de domínios alcançados nesta nova infraestrutura fornecem uma base tecnológica extremamente sólida para a contínua expansão do produto. A separação do *Schedule Service* e do *User Service*, por exemplo, viabiliza e facilita os fluxos transacionais de pagamentos utilizando a API do Mercado Pago, bem como o uso de painéis analíticos avançados para apoiar a tomada de decisão estratégica dos proprietários.

Em suma, o desenvolvimento e a modernização arquitetural desta aplicação representam um passo decisivo em direção à transformação digital e ao aumento da competitividade no setor de estética. O projeto demonstra na prática como a adoção de padrões de

engenharia de *software* modernos e infraestruturas escaláveis em nuvem resolve desafios operacionais diários, entregando valor contínuo aos utilizadores finais.

REFERÊNCIAS

- [1] CARVALHO, Marcella Souto; TOZI, Luiz Antonio. *Uso de tecnologia como apoio a atividades de gestão em pequenos negócios: caso da barbearia*. Acesso em: 29/08/2025.
- [2] CLOUDINARY. *Java Integration*. Santa Clara, CA, [c2024]. Disponível em: https://cloudinary.com/documentation/java_integration. Acesso em: 29/08/2025.
- [3] IDEAL SALES. *Como O CRM Pode Fidelizar Clientes E Aumentar O Faturamento Da Sua Barbearia*. 2025. Disponível em: <https://www.idealsales.com.br/blog/como-o-crm-pode-fidelizar-clientes-e-aumentar-o-faturamento-da-sua-barbearia/>. Acesso em: 12/09/2025.
- [4] IJIRT. *Design and Implementation of an Online Appointment System for Barber Shops*. [S.l.], [s.d.]. Disponível em: https://ijirt.org/publishedpaper/IJIRT177309_PAPER.pdf. Acesso em: 07/09/2025.
- [5] IJRASET. *Barber Shop Management System*. 2023. Disponível em: <https://www.ijraset.com/best-journal/barber-shop-management-system>. Acesso em: 18/09/2025.
- [6] MUTUAL TECNOLOGIA. *Inovações em Agendamentos para Barbearias*. 2025. Disponível em: <https://mutualtecnologia.com/inovacoes-em-agendamentos-para-barbearias/>. Acesso em: 12/09/2025.
- [7] ORACLE. *Java SE Documentation*. Redwood Shores, CA, [c2024]. Disponível em: <https://docs.oracle.com/en/java/>.
- [8] PROJECT LOMBOK. *Features*. [S.l.]: [c2024]. Disponível em: <https://projectlombok.org/features/>.
- [9] REVISTA CIENTÍFICA DA UNIFENAS. *APLICATIVO. SALÃO DE BELEZA. BARBEARIA. AGENDAMENTO ONLINE*. 2023. Disponível em: <https://revistas.unifenas.br/index.php/revistaunifenas/article/view/875>. Acesso em: 08/10/2025.
- [10] SEBRAE. *Softwares e aplicativos facilitam e melhoram a gestão do salão*. 2023. Disponível em: <https://sebrae.com.br/sites/PortalSebrae/artigos/softwares-e-aplicativos-facilitam-e-melhoram-a-gestao-do-salao,f06cac941b896810VgnVCM1000001b00320aRCRD>. Acesso em: 12/09/2025.

-
- [11] SPRING. *Spring Boot Reference Documentation*. [S.l.]: VMware, [c2024]. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. Acesso em: 29/08/2025.
- [12] SPRING. *Spring Data JPA Reference Documentation*. [S.l.]: VMware, [c2024]. Disponível em: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>. Acesso em: 01/09/2025.
- [13] SPRING. *Spring Security Documentation*. [S.l.]: VMware, [c2024]. Disponível em: <https://docs.spring.io/spring-security/reference/index.html>. Acesso em: 01/09/2025.
- [14] SPRINGDOC. *Springdoc-OpenAPI*. [S.l.]: [c2024]. Disponível em: <https://springdoc.org/>. Acesso em: 02/09/2025.
- [15] TRINKS. *Agenda online para barbearia: o que é e como funciona?*. 2025. Disponível em: <https://blog.trinks.com/agenda-online-para-barbearia-o-que-e-e-como-funciona/>. Acesso em: 07/09/25.
- [16] DOCKER. *Docker Documentation*. [S.l.]: Docker Inc., [c2026]. Disponível em: <https://docs.docker.com/>. Acesso em: 05/02/2026.
- [17] GOOGLE DEVELOPERS. *Progressive Web Apps (PWAs)*. [S.l.]: Google, [c2026]. Disponível em: <https://developer.google.com/web/progressive-web-apps>. Acesso em: 14/04/2026.
- [18] FOWLER, Martin; LEWIS, James. *Microservices: a definition of this new architectural term*. [S.l.]: MartinFowler.com, [c2014]. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 04/02/2026.
- [19] MOUAT, Adrian. *Uso do Docker: Otimize o Desenvolvimento, a Entrega e a Implantação de Seus Aplicativos*. São Paulo: Novatec Editora, 2016.
- [20] NEWMAN, Sam. *Building Microservices: Designing Fine-Grained Systems*. 2. ed. Sebastopol: O'Reilly Media, 2021.
- [21] REDIS. *Redis Documentation*. [S.l.]: Redis Ltd., [c2026]. Disponível em: <https://redis.io/docs/>. Acesso em: 25/03/2026.
- [22] SPRING. *Spring Cloud Gateway Reference Documentation*. [S.l.]: VMware, [c2026]. Disponível em: <https://spring.io/projects/spring-cloud-gateway>. Acesso em: 16/02/2026.
- [23] WALLS, Craig. *Spring in Action*. 6. ed. Shelter Island: Manning Publications, 2022.

APÊNDICE A – DICIONÁRIO DE DADOS

TABELA: BARBERSHOPS

Descrição: Armazena informações sobre os estabelecimentos (barbearias) cadastrados.

id: VARCHAR(36). Chave Primária (PK). Identificador único universal (UUID) da barbearia.

name: VARCHAR(255). Obrigatório (NOT NULL). Nome comercial da barbearia.

cnpj: VARCHAR(14). Obrigatório, Único (UNIQUE). Número do CNPJ.

address: VARCHAR(255). Endereço completo da barbearia.

date_created: DATETIME. Obrigatório. Data e hora de criação do registro.

last_updated: DATETIME. Obrigatório. Data e hora da última atualização.

logo_url: VARCHAR(255). URL para o arquivo de logo da barbearia.

banner_url: VARCHAR(255). URL para o arquivo de banner da barbearia.

TABELA: BARBERS

Descrição: Armazena informações dos barbeiros (incluindo proprietários).

id: VARCHAR(36). Chave Primária (PK). Identificador único (UUID) do barbeiro.

name: VARCHAR(70). Obrigatório. Nome completo do barbeiro.

tell: VARCHAR(11). Obrigatório, Único. Número de telefone/celular.

email: VARCHAR(70). Obrigatório, Único. Endereço de e-mail (usado para login).

document_cpf: VARCHAR(11). Obrigatório, Único. Número do CPF.

password: VARCHAR(255). Obrigatório. Senha criptografada (hash).

barbershop_id: VARCHAR(36). Chave Estrangeira (FK) de barbershops. Indica a barbearia vinculada. Permite NULL.

is_owner: TINYINT(1). Obrigatório. Flag booleana (1 para proprietário, 0 para funcionário).

date_created: DATETIME. Obrigatório. Data e hora de criação.

last_updated: DATETIME. Obrigatório. Data e hora da última atualização.

work_start_time: TIME. Hora de início da jornada de trabalho.

work_end_time: TIME. Hora de término da jornada de trabalho.

image_url: VARCHAR(255). URL para a foto de perfil do barbeiro.

TABELA: CUSTOMERS

Descrição: Armazena informações dos clientes usuários da plataforma.

id: VARCHAR(36). Chave Primária (PK). Identificador único (UUID) do cliente.

name: VARCHAR(70). Obrigatório. Nome completo do cliente.

tell: VARCHAR(11). Obrigatório, Único. Número de telefone/celular.

email: VARCHAR(70). Obrigatório, Único. Endereço de e-mail (usado para login).

document_cpf: VARCHAR(11). Obrigatório, Único. Número do CPF.

password: VARCHAR(255). Obrigatório. Senha criptografada (hash).

date_created: DATETIME. Obrigatório. Data e hora de criação.

last_updated: DATETIME. Obrigatório. Data e hora da última atualização.

image_url: VARCHAR(255). URL para a foto de perfil do cliente.

TABELA: ACTIVITIES

Descrição: Armazena a lista de serviços oferecidos por uma barbearia.

id: VARCHAR(36). Chave Primária (PK). Identificador único (UUID) da atividade/serviço.

barbershop_id: VARCHAR(36). Chave Estrangeira (FK) de barbershops. Obrigatório.

activity_name: VARCHAR(255). Obrigatório. Nome do serviço (Ex.: "Corte de Cabelo").

price: DECIMAL(10,2). Obrigatório. Preço do serviço.

duration_minutes: INT. Obrigatório. Duração estimada do serviço em minutos.

date_created: DATETIME. Obrigatório. Data e hora de criação.

last_updated: DATETIME. Obrigatório. Data e hora da última atualização.

image_url: VARCHAR(255). URL para uma imagem representativa do serviço.

TABELA: APPOINTMENTS

Descrição: Armazena o registro de cada agendamento.

id: BIGINT. Chave Primária (PK), Auto Incremento (AI).

barbershop_id: VARCHAR(36). Chave Estrangeira (FK) de barbershops. Obrigatório.

barber_id: VARCHAR(36). Chave Estrangeira (FK) de barbers. Obrigatório.

customer_id: VARCHAR(36). Chave Estrangeira (FK) de customers. Obrigatório.

start_time: DATETIME. Obrigatório. Data e hora de início do agendamento.

end_time: DATETIME. Obrigatório. Data e hora de término do agendamento.

status: VARCHAR(50). Obrigatório (Padrão: SCHEDULED).

date_created: DATETIME. Obrigatório. Data e hora de criação.

last_updated: DATETIME. Obrigatório. Data e hora da última atualização.

TABELA: APPOINTMENT_ACTIVITIES

Descrição: Tabela de ligação N:M (agendamentos ↔ activities).

id: BIGINT. Chave Primária (PK), Auto Incremento (AI).

appointment_id: BIGINT. Chave Estrangeira (FK) de appointments. Obrigatório.

activity_id: VARCHAR(36). Chave Estrangeira (FK) de activities.

TABELA: BARBER_ACTIVITIES

Descrição: Tabela de ligação N:M (barbers ↔ activities).

barber_id: VARCHAR(36). Chave Primária Composta, FK de barbers. Obrigatório.

activity_id: VARCHAR(36). Chave Primária Composta, FK de activities. Obrigatório.

TABELA: BARBERSHOP_JOIN_REQUESTS

Descrição: Armazena os pedidos de barbeiros para se vincular a uma barbearia.

id: BIGINT. Chave Primária (PK), Auto Incremento (AI).

barber_id: VARCHAR(36). Chave Estrangeira (FK) de barbers. Obrigatório.

barbershop_id: VARCHAR(36). Chave Estrangeira (FK) de barbershops. Obrigatório.

status: ENUM('PENDING','REJECTED'). Obrigatório (Padrão: PENDING).

date_created: DATETIME. Obrigatório. Data e hora da solicitação.

TABELA: BARBERSHOP_HIGHLIGHTS

Descrição: Armazena imagens de destaque ou portfólio de uma barbearia.

id: VARCHAR(36). Chave Primária (PK). Identificador único (UUID) do destaque.

image_url: VARCHAR(255). Obrigatório. URL para a imagem de destaque.

barbershop_id: VARCHAR(36). Chave Estrangeira (FK) de barbershops. Obrigatório.